COORDENADORIA DE ENGENHARIA DA COMPUTAÇÃO

ARNALD SOUZA

BRUNO GABRIEL DE OLIVEIRA TARGA

VICTOR SOARES NUNES PIRES DE OLIVEIRA

APLICAÇÃO DA VISÃO COMPUTACIONAL PARA A DETECÇÃO DE PLACAS DE VEÍCULOS ROUBADOS

SOROCABA, SP

2025

ARNALD SOUZA

BRUNO GABRIEL DE OLIVEIRA TARGA

VICTOR SOARES NUNES PIRES DE OLIVEIRA

APLICAÇÃO DA VISÃO COMPUTACIONAL PARA A DETECÇÃO DE PLACAS DE VEÍCULOS ROUBADOS

Trabalho de conclusão de curso apresentado ao Centro Universitário Facens como exigência parcial para obtenção do diploma de graduação em Engenharia da Computação.

Orientador (a): Prof. Allan Marconato Marum

SOROCABA, SP
2025

FICHA CATALOGRÁFICA ELABORADA

PELA "BIBLIOTECA FACENS"

S729a

Souza, Arnald.

A questão da sustentabilidade na era do consumo / por Rodrigo Guimarães Campos, Diogo Satilio Carvalho Pinho, Jéssica Martins Vieira – Sorocaba, SP: [s.n.], 2020.

32f.

Trabalho de Conclusão de Curso (Graduação) – Centro Universitário Facens – Curso de Engenharia Civil, 2020.

Orientador: Prof.(a) Dr.(a) Eliane da Rocha

1. Sustentabilidade. 2. Era do consumo. 3. Consumismo.

I. Pinho, Diogo Satilio Carvalho. II. Vieira, Jéssica Martins. III. Centro Universitário Facens. IV. Título.

ARNALD SOUZA

BRUNO GABRIEL DE OLIVEIRA TARGA

VICTOR SOARES PIRES NUNES DE OLIVEIRA

APLICAÇÃO DA VISÃO COMPUTACIONAL PARA A DETECÇÃO DE PLACAS DE VEÍCULOS ROUBADOS

Trabalho de conclusão de curso apresentado ao Centro Universitário Facens como exigência parcial para obtenção do certificado de conclusão do curso de Engenharia da Computação.
Orientador (a): Prof. Allan Marconato Marum

BANCA EXAMINADORA

Prof.
Afiliação (Centro Universitário Facens)

Prof.
Afiliação (Centro Universitário Facens)

Prof.ª
Afiliação (Centro Universitário Facens)

Dedico este trabalho a todos que fizeram parte desta jornada de desenvolvimento profissional, e aos que contribuíram, direta ou indiretamente, para que meu objetivo fosse alcançado com sucesso, em meio a um processo permeado por alegrias.

AGRADECIMENTOS

Agradeço a esta instituição de ensino por me preparar para o mercado de trabalho, a todos os professores, coordenadores, colegas de classe e funcionários em geral que de alguma forma estiveram comigo nesta trajetória.

Aos meus familiares, que depositaram muita confiança e credibilidade em mim e em meus planos.

Aos orientadores pelo incentivo disponibilizado, ao tempo doado para que eu pudesse encontrar a melhor forma de concluir o meu projeto.

A todos integrantes da biblioteca da faculdade pela atenção e carinhoso suporte.

E, em especial, a agência fomentadora XYZ pelo apoio financeiro que me propiciou uma excelente pesquisa, para que eu pudesse encontrar os resultados desejados.

Meus sinceros agradecimentos.

“A tribulação produz perseverança; a perseverança, experiência; e a experiência, esperança.**”.**

(Romanos 5:3-4)

RESUMO

Questão antiga e que é refletida de forma mais incessante nos dias atuais: sustentabilidade - diante do aumento do consumo/consumismo. Apresenta-se, no decorrer do trabalho, as consequências do consumo desregulado e os fatores que são impactados diretamente. Em contrapartida, elaborou-se algumas práticas corretas que podem ser adotadas no dia a dia para amenizar as mazelas ao meio ambiente como um todo e tornar o 1º R – Repensar - um ato cotidiano antes de realizar as compras. Mostra a diferença entre ação de consumir e o ato do consumismo. Tem-se como objetivo explicitar a problemática do equilíbrio no ato de comprar em prol da proteção ambiental. Para o desenvolvimento do texto, utilizou-se a metodologia de levantamento bibliográfico cujos textos têm a mesma perspectiva com um recorte no tempo de publicação da pesquisa últimos 10 anos. A conclusão está voltada à redução de compras não necessárias e da aquisição de hábitos conscientes, que provoquem, em si e no outro, uma maior valorização dos recursos naturais.

Palavras-chave: Sustentabilidade. Consumismo. Consumo Consciente.

ABSTRACT

An old issue that is reflected more incessantly nowadays: sustainability - in the face of increased consumption/consumerism. Throughout the work, the consequences of unregulated consumption and the factors that are directly impacted are presented. On the other hand, some correct practices were elaborated that can be adopted on a daily basis to mitigate the damage to the environment as a whole and make the 1st R - Rethink - a daily act before making purchases. It shows the difference between the action of consuming and the act of consumerism. It aims to explain the problem of balance in the act of buying for the sake of environmental protection. For the development of the text, we used the methodology of bibliographic survey whose texts have the same perspective with a cut in the time of publication of the research last 10 years. The conclusion is focused on the reduction of unnecessary purchases and the acquisition of conscious habits, which cause, in oneself and in others, a greater appreciation of natural resources.

Keywords: Sustainability. Consumerism. Conscious Consumption.

LISTA DE ILUSTRAÇÕES

Figura 1 - Tipos de Aprendizado de Máquina.....19
Figura 2 - Representações de Redes Neurais 20
Figura 3 - Arquitetura de uma CNN.....21
Figura 4 - Aplicação de Bounding Boxes 22
Figura 5 - Tipos de Segmentação 22
Figura 6 - Pipeline de Treinamento do YOLO 35
Figura 7 - Definição do Modelo e Parâmetros.....36
Figura 8 - Treino e Inferência 36
Figura 9 - Configurações das Funções do OpenCV.....37
Figura 10 - Fluxograma da Detecção OCR 38
Figura 11 - Pipeline de Detecção OCR 39
Figura 12 - Desempenho Comparativo dos Modelos 47
Figura 13 - Curvas de Aprendizado YOLO.....47
Figura 14 - Curvas de Aprendizado FASTER R-CNN 48
Figura 15 - Curvas PR.....48
Figura 16 - Placa Detectada.....49
Figura 17 - Placa não Detectada.....49

LISTA DE SIGLAS

IA	Inteligência Artificial
CNN	Convolutional Neural Network
R-CNN	Region-Convolutional Neural Network
KNN	K–Nearest Neighbors
ReLU	Rectified Linear Unit
RPN	Region Proposal Network
YOLO	You Only Look Once
OCR	Optical Character Recognition
HOG	Histogram of Oriented Gradients
SVM	Support Vector Machine
SIFT	Scale Invariant Featur Transform
SURF	Speeded Up Robust Features
ROI	Region of Interest
FPN	Feature Pyramid Networks
OpenCV	Open Source Computer Vision

SUMÁRIO

1 INTRODUÇÃO 14
2 VISÃO COMPUTACIONAL 16
2.1 TÉCNICAS CLÁSSICAS E FERRAMENTAS 16
2.2 PROCESSOS DE DETECÇÃO E TRANSFORMAÇÕES 17
2.3 APRENDIZADO DE MÁQUINA 18
2.4 APRENDIZADO PROFUNDO 20
2.5 REDES NEURAIS CONVOLUCIONAIS 20
2.5.1 Aplicações em detecção de objetos e segmentação de imagens 21
2.6 ARQUITETURAS DE MODELOS UTILIZADAS **23**
2.6.1 YOLO (You Only Look Once) 24
2.6.2 Faster R-CNN e o dataset COCO 25
2.6.3 OpenCV e OCR 26
3 METODOLOGIA 28
3.1 ABORDAGEM DO ESTUDO 28
3.1.1 Tipo de pesquisa 28
3.1.2 Objetivo do modelo 29
3.2 CONJUNTO DE DADOS 30
3.2.1 Principal *dataset* utilizado 30
3.2.2 Fontes adicionais e dados abertos 31
3.2.3 Divisão dos dados 31
3.3 TÉCNICAS DE PRÉ-PROCESSAMENTO DE DADOS 31
3.3.1 Redimensionamento e padronização das imagens 32
3.3.2 Enriquecimento de dados 32
4 IMPLEMENTAÇÃO 33
4.1 CONFIGURAÇÃO DO AMBIENTE 33
4.1.1 Ferramentas e bibliotecas utilizadas 33
4.1.2 Hardware e recursos computacionais 34
4.2 IMPLEMENTAÇÃO DAS ARQUITETURAS 34
4.2.1 YOLO 34
4.2.2 Faster R-CNN 35
4.2.3 OpenCV 37

4.2.4 EasyOCR 38
4.3 PROCESSO DE TREINAMENTO 39
4.3.1 Ajuste de hiperparâmetros 39
4.3.2 Estratégia de validação 40
5 AVALIAÇÃO E RESULTADOS 42
5.1 MÉTRICAS PARA YOLO E FASTER R-CNN 42
5.1.3 F1-Score 43
5.1.4 *Mean Average Precision* (mAP) 43
5.2 MÉTRICAS PARA OCR 44
5.2.1 *String accuracy* 44
5.2.2 *Character error rate* 44
5.3 RESULTADOS OBTIDOS 45
5.4 ANÁLISE COMPARATIVA 51
5.4.1 Discussão sobre vantagens e limitações 51
6 CONSIDERAÇÕES FINAIS 53
REFERÊNCIAS 54

1 INTRODUÇÃO

Sabe-se que a inteligência artificial é uma das tecnologias que mais se tornou popular nos últimos anos, principalmente por sua capacidade computacional, sendo usada em diversas áreas, como no transporte, saúde, educação, segurança e muito mais (Jiang *et al.*, 2022). Quando aplicada na educação ela é utilizada como um otimizador de busca e como um facilitador de estudos.

A visão computacional é um ramo da inteligência artificial (IA) que permite que computadores e sistemas "enxerguem", interpretem e compreendam o mundo visual. Um dos objetivos deste trabalho é mostrar a possibilidade do uso da visão computacional para a segurança pública, focando na recuperação de carros roubados. As formas tradicionais de lidar com esse problema nem sempre são as melhores, tornando o processo demorado e pouco eficiente. Isso demonstra como é necessário e importante o uso da tecnologia para melhorar a forma de como os carros são identificados, por consequência aumentando a recuperação de veículos roubados (Andrade *et al.*, 2023).

Para ilustrar um exemplo da implementação na segurança pública, podemos mostrar o caso da Prefeitura da cidade de São Paulo que implantou o *Smart Sampa*:

> Mais uma das iniciativas da gestão municipal em reforçar a segurança da cidade. O objetivo é garantir uma pronta resposta na segurança à população de São Paulo. O Programa prevê integração com diversos outros órgãos, como SAMU, Corpo de Bombeiros, dentre outros. (Prefeitura de São Paulo, 2025).

Conforme apresentado na reportagem exibida pela equipe da Band (2025):

> O programa da prefeitura de São Paulo ajudou na prisão de mil criminosos que estavam foragidos da Justiça, entre outubro do ano passado, quando o sistema passou efetivamente a fazer a contagem, e esta segunda-feira (14).

Nesse cenário, a área de visão computacional atende o ponto que é necessário, pois é possível analisar de maneira automatizada imagens das câmeras de segurança e identificar a placa de veículos com irregularidades. Portanto, é necessário para um bom funcionamento tratar algumas lacunas, como melhoria de precisão, tratamento de imagens e integração a programas já existentes.

Este trabalho busca implementar um modelo de visão computacional para identificação de veículos roubados. Para isso, primeiramente, serão explorados os

conceitos da visão computacional e suas aplicações nesse contexto. Em seguida, será desenvolvido o modelo capaz de reconhecer veículos roubados por meio da análise de imagens capturadas por câmeras de monitoramento.

Após a implementação, serão realizados testes para validar a precisão e a eficiência do modelo proposto. O desenvolvimento deste estudo será feito de maneira quantitativa e experimental, usando modelos de visão computacional focados em identificar veículos roubados. Sendo assim, serão utilizadas redes neurais convolucionais (*Convolutional Neural Network - CNNs*), técnicas de enriquecimento de dados e métodos de avaliação de desempenho.

Neste estudo, será implementado um modelo em cascata composto por três etapas. Inicialmente, o YOLO (Redmond; Divvala, 2016) é responsável por realizar uma detecção rápida das placas dos veículos, priorizando a agilidade do processo. Em seguida, as regiões com menor confiabilidade são refinadas pelo Faster R-CNN (Ren; He; Girshick; Sun, 2015), que tem como objetivo aumentar a precisão e reduzir erros de detecção. Por fim, as regiões validadas passam por um módulo de OCR (*Optical Character Recognition* - Reconhecimento Óptico de Caracteres), encarregado de extrair o conteúdo alfanumérico das placas, completando assim o processo de identificação automática. Essa abordagem híbrida busca equilibrar velocidade, confiabilidade e capacidade de leitura, integrando detecção e reconhecimento em um único pipeline.

O treinamento e a validação do modelo serão realizados com dados de fontes abertas, possibilitando maior diversidade e abrangência nos exemplos analisados, ajudando a reduzir custos operacionais. Os desempenhos serão comparados por meio de gráficos e tabelas, possibilitando uma análise detalhada.

O objetivo final deste estudo é propor um sistema em cascata que una rapidez e precisão na detecção e reconhecimento de placas veiculares, integrando YOLO, Faster R-CNN e OCR em um fluxo único. A justificativa para essa abordagem está na possibilidade de equilibrar desempenho em tempo real com confiabilidade, viabilizando uma solução de baixo custo baseada em dados abertos e com potencial aplicação em segurança pública e monitoramento inteligente, especialmente no auxílio à identificação e rastreamento de veículos roubados.

2 VISÃO COMPUTACIONAL

De acordo com Szeliski (2022), a visão computacional pode ser compreendida como o campo da ciência da computação dedicado a desenvolver algoritmos e técnicas capazes de extrair, analisar e compreender informações úteis a partir de imagens e vídeos. Seu objetivo central é transformar dados visuais em representações estruturadas, permitindo aplicações que vão desde o reconhecimento de objetos e reconstrução tridimensional até sistemas de visão embarcados em veículos autônomos.

O foco da visão computacional neste projeto está na conversão de dados visuais, geralmente obtidos por meio de câmeras, em descrições computacionais, permitindo a realização de tarefas como reconhecimento de padrões, detecção de objetos e rastreamento de movimento.

2.1 TÉCNICAS CLÁSSICAS E FERRAMENTAS

Há diversas técnicas de visão computacional para a extração e análise de características em imagens. Entre as principais abordagens, destaca-se a detecção de bordas, que utiliza operadores como Sobel, Canny e Laplaciano para identificar contornos e mudanças de intensidade, utilizadas para a segmentação de objetos. Outra técnica também usada é a transformada de Hough, empregada na identificação de formas geométricas, como linhas e círculos, facilitando a detecção de estruturas regulares em imagens. A segmentação por limiarização (*thresholding*) também é aplicada para separar regiões de interesse a partir de um valor de intensidade (Nixon e Aguado, 2019; Benhajyoussef e Saidani, 2024).

O Histograma de Gradientes Orientados (HOG - *Histogram of Oriented Objects*) constitui uma técnica de extração de características que explora informações estruturais a partir da orientação dos gradientes. Conforme descreve Szeliski (2022, pp. 376–377), "um exemplo de um detector de pedestres bem conhecido é o algoritmo desenvolvido por Dalal e Triggs (2005), que utiliza um conjunto de descritores de histogramas de gradientes orientados (HOG) sobrepostos, fornecidos a uma máquina de vetores de suporte (SVM - *Support Vector Machine*)". Dessa forma, a técnica

fundamenta-se na distribuição estatística dos gradientes locais e é amplamente aplicada em detectores de pedestres.

Além dessa técnica, temos os algoritmos SIFT (*Scale-Invariant Feature Transform* – Transformada de Características Invariantes à Escala), proposta por Lowe (2004), que detecta pontos-chave em imagens e gera descritores invariantes a escala, rotação e ruído, sendo amplamente utilizado em reconhecimento de objetos, correspondência e recuperação de imagens. Também o SURF (*Speeded - Up Robust Features* – Características Robustas Aceleradas), uma alternativa ao SIFT, em que foi projetado por Bay, Van Gool e Tuytelaar (2006) para ser mais rápido, preservando a robustez em relação a mudanças de iluminação, escala e rotação (Riabko; Averyanova, 2024).

Entre as principais ferramentas utilizadas para a implementação de técnicas de visão computacional, destaca-se a OpenCV, uma biblioteca *open source* amplamente adotada tanto em projetos acadêmicos quanto na indústria (OpenCV, 2025). Em projetos desenvolvidos com a linguagem Python, destacam-se as bibliotecas scikit-image e Pillow (PIL), que oferecem recursos eficientes para o pré-processamento e manipulação de imagens, como ajustes de tamanho, filtros e operações de transformação (Scikit-image, 2025). Para integrações mais avançadas com técnicas de aprendizado profundo, frameworks como TensorFlow, PyTorch e Keras são amplamente empregados, pois permitem o desenvolvimento, o treinamento e a implementação de modelos de redes neurais voltados para aplicações de visão computacional, combinando flexibilidade, eficiência e escalabilidade.

2.2 PROCESSOS DE DETECÇÃO E TRANSFORMAÇÕES

O processo de detecção de objetos por meio da visão computacional envolve uma sequência estruturada de etapas, vai desde o pré-processamento da imagem até a classificação dos objetos. O pré-processamento consiste em operações como redimensionamento, normalização, redução de ruído e realce de contraste, com o objetivo de melhorar a qualidade das imagens e, consequentemente, a eficiência das etapas posteriores de análise (Kang *et al.*, 2025)

Em seguida, temos a detecção de regiões de interesse (ROI – *Region of Interest*), utilizando técnicas como *sliding window* ou algoritmos de propostas de

regiões, como o *Selective Search* e as *Region Proposal Networks* – RPN, que permitem identificar localizações prováveis de objetos na imagem (Ren *et a*l., 2015). Estudos comparativos mostram que métodos baseados em RPN tendem a manter *recall* elevado mesmo quando o número de propostas é reduzido, enquanto métodos como *selective search* ou *sliding window* têm queda acentuada de desempenho nessas condições (Ghodrati *et al.*, 2017).

Para tornar os modelos mais robustos às variações do ambiente real, são aplicadas transformações geométricas, como rotações, escalas, espelhamentos e translações. Essas transformações também são incorporadas na etapa de enriquecimento dados, que gera novas amostras de treinamento a partir das imagens originais, contribuindo para reduzir o risco de *overfitting* e melhorar a capacidade de generalização dos modelos (Shorten; Kho, 2019).

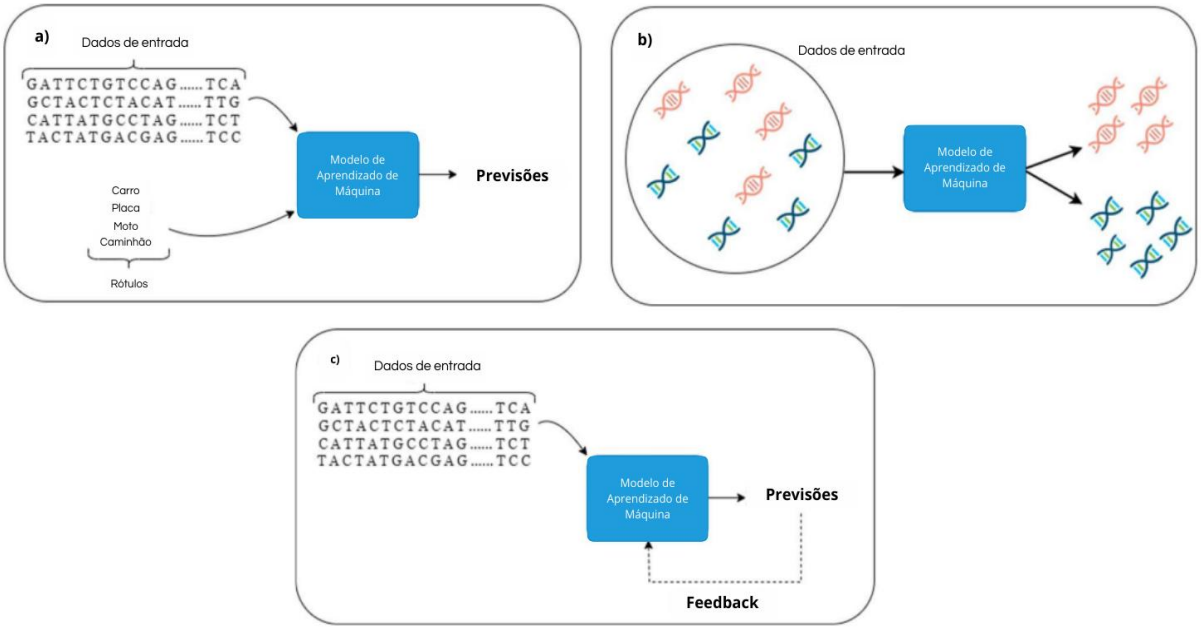
Na etapa de extração de características, redes neurais convolucionais (CNNs) são responsáveis por identificar automaticamente padrões relevantes nas regiões de interesse, eliminando a necessidade de descritores manuais como bordas e texturas. Essa capacidade de aprender representações hierárquicas dos dados é um dos principais fatores que impulsionaram os avanços recentes na área. Posteriormente, a classificação e a detecção dos objetos são realizadas por arquiteturas específicas, como YOLO e Faster R-CNN, se destacando pela eficiência em localizar e reconhecer objetos por meio do uso de caixas delimitadoras (Kang *et al.*, 2025).

2.3 APRENDIZADO DE MÁQUINA

O conceito de aprendizado pode ser compreendido como:

> "o processo de extrair conhecimento a partir de dados. Trata-se de um campo de pesquisa na interseção da estatística, da inteligência artificial e da ciência da computação, também conhecido como análise preditiva ou aprendizado estatístico." (Müller; Guido, 2017, p.1, tradução própria).

Figura 1 - Tipos de Aprendizado de Máquina



Fonte: adaptado de Alatrany, 2022

A Figura 1 nos demonstra como podem ser divididos os tipos de aprendizado de máquina, no primeiro item o aprendizado supervisionado, que é caracterizado pelo uso de dados rotulados, nos quais cada entrada está associada a uma saída esperada. Nesse caso, o modelo é treinado a partir desses exemplos para aprender padrões e, em seguida, ser capaz de prever a saída de novos dados não observados. Entre os principais algoritmos utilizados neste contexto estão a regressão linear, a regressão logística, as árvores de decisão, o k-nearest neighbors (KNN) e as redes neurais (Faouzi, 2023).

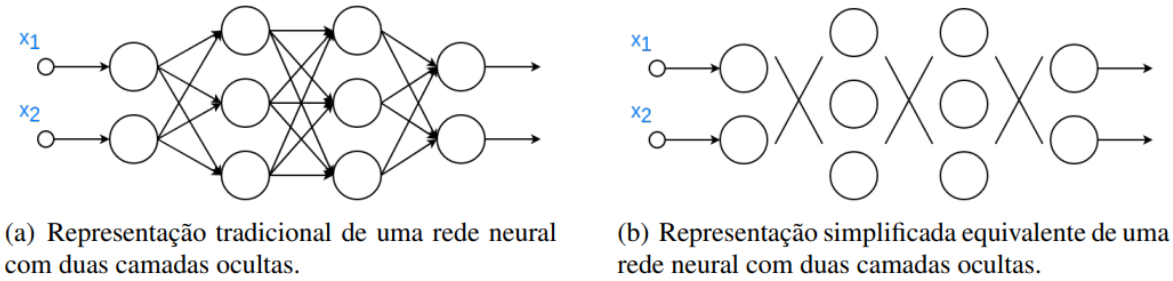
No segundo item, o aprendizado não supervisionado é aquele que recebe dados, mas sem rótulos, e ele tenta descobrir os padrões por conta própria. Os principais modelos utilizados são o agrupamento *k-Means*, *DBSCAN* e a associação a priori (Mukhopadhyay; Samanta, 2022).

Por fim, o último item mostra que o aprendizado por reforço é aquele em que consiste em fornecer dados não rotulados ao modelo, que produz previsões que são posteriormente validadas através de *feedback*, indicando a precisão de cada previsão (Alatrany *et. al,* 2022).

2.4 APRENDIZADO PROFUNDO

O aprendizado profundo, é um ramo do aprendizado de máquina que trabalha, de forma eficiente, com grandes volumes de dados e tarefas complexas, como reconhecimento de imagens, fundamenta-se em modelos computacionais conhecidos como redes neurais, os quais são inspirados no processamento de informações e nos processos de aprendizado do cérebro humano (Bishop; Bishop, 2024).

Figura 2 - Representações de Redes Neurais



(a) Representação tradicional de uma rede neural com duas camadas ocultas.

(b) Representação simplificada equivalente de uma rede neural com duas camadas ocultas.

Fonte: Bochie *et al.*, 2020

O principal componente do aprendizado profundo são as redes neurais profundas, como pode-se analisar na Figura 2, o item a mostra uma rede neural tradicional, compostas por duas camadas ocultas de neurônios. Já o item b mostra uma rede neural equivalente. Como afirma Bochie *et al.* (2020), sobre as camadas das redes neurais:

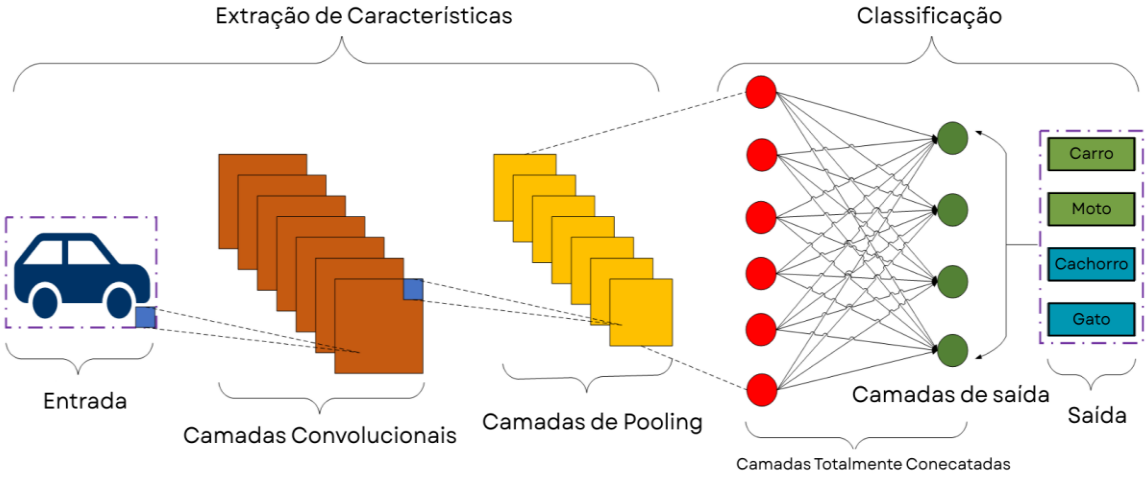
> As camadas são organizações paralelas das estruturas de processamento básicas, que permitem a extração de informações progressivamente mais complexas à medida em que os dados são processados através das múltiplas camadas intermediárias, de forma hierárquica.

2.5 REDES NEURAIS CONVOLUCIONAIS

As redes neurais convolucionais (CNNs) permitem detectar padrões de forma eficiente, aproveitando o conceito de campos receptivos locais (*local receptive fields*), ao contrário das redes totalmente conectadas, nas quais cada neurônio recebe entrada de todos os neurônios da camada anterior. A arquitetura típica de uma CNN inclui camadas convolucionais (que aplica filtros às regiões locais da entrada), funções de ativação (comumente ReLU), camadas de *pooling* (para subamostragem espacial) e camadas densas no final, para realizar a decisão ou classificação (Raj; Kos,2025).

A Figura 3 apresenta a arquitetura básica de uma CNN, composta por duas partes: extração de características e classificação. Na extração de características, a entrada (imagem de um carro) passa por camadas convolucionais, que extraem informações, e por camadas de *pooling*, que reduzem a dimensionalidade. Na classificação, os dados são processados por camadas totalmente conectadas, que integram as características para gerar a saída. Esta é composta por quatro nós verdes, cada um associado a uma classe prevista: carro, moto, cachorro e gato.

Figura 3 - Arquitetura de uma CNN



Fonte: adaptado de Raj;Kos, 2025

Essa estrutura hierárquica de camadas permite aprender representações cada vez mais abstratas a partir de padrões simples, tornando as CNNs especialmente eficazes em tarefas de visão computacional e percepção robótica (Raj;Kos, 2025).

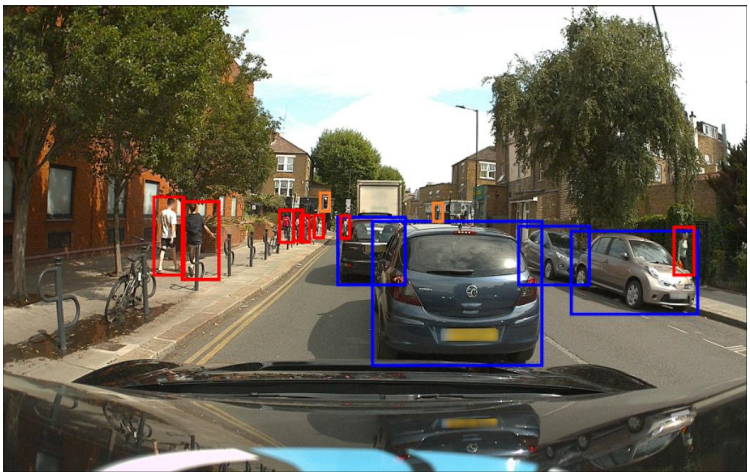
2.5.1 Aplicações em detecção de objetos e segmentação de imagens

De acordo com Li e Huang (2024), a detecção e a segmentação de objetos têm recebido grande atenção e sido alvo de diversas pesquisas, uma vez que a visão computacional tem proporcionado inúmeras oportunidades e aplicações em vários campos tecnológicos.

A detecção de objetos refere-se ao processo de identificar a presença de objetos de interesse em uma imagem e determinar suas localizações exatas, geralmente através da definição de caixas delimitadoras (Zhao *et al.*, 2019). Esta tarefa não apenas reconhece quais objetos estão presentes, mas também onde eles

se encontram na cena, como podemos ver na Figura 4, que temos caixas delimitadoras em volta dos carros, pessoas e do semáforo.

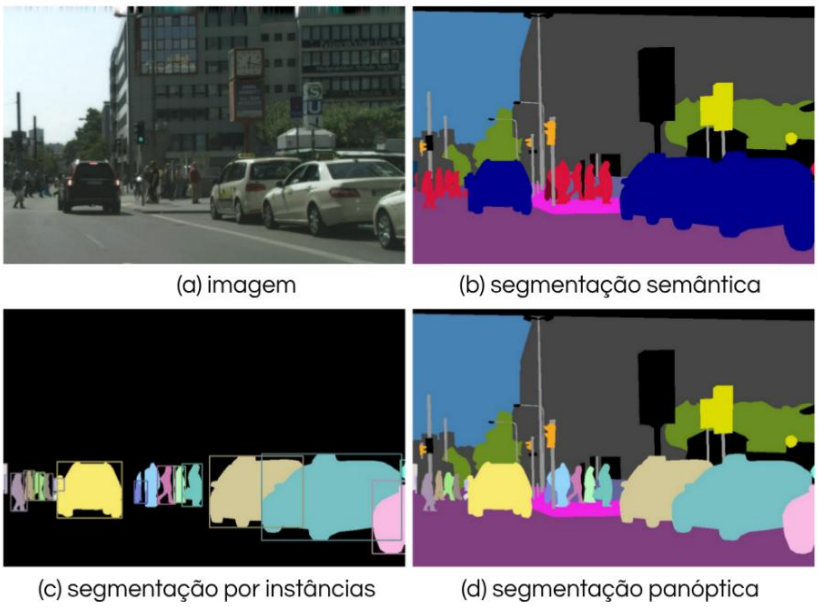
Figura 4 - Aplicação de *Bounding Boxes*



Fonte: Bishop;Bishop, 2024.

Como afirma Zhou *et al.* (2024), a segmentação de imagens é um campo central e desafiador da visão computacional, voltado a agrupar pixels em regiões distintas, servindo como etapa inicial para tarefas de maior nível, com aplicações em áreas como direção autônoma, análise médica, vigilância e edição de imagens.

Figura 5 - Tipos de Segmentação



Fonte: adaptado de Kirillov *et al., 2018*

A segmentação de imagens é subdividida em três tipos, como pode-se ver na Figura 5. O item a, mostra a imagem original. A segmentação semântica, item b, busca

rotular cada pixel de uma imagem de acordo com a classe a que pertence, de modo que todos os pixels da mesma categoria recebam a mesma marcação, sem distinguir entre diferentes instâncias de um mesmo objeto. Dessa forma, a segmentação semântica relaciona-se diretamente com técnicas de detecção de objetos, mas com o foco em atribuir rótulos a todos os pixels da cena (Liu *et al.*, 2019; Csurka *et al.,* 2023).

Na segmentação por instâncias, item c, ocorre um avanço em relação à segmentação semântica, pois além de reconhecer os objetos por classe, também diferencia cada instância de forma individual, mesmo quando pertencem à mesma categoria. Em outras palavras, essa abordagem pode ser entendida como a combinação de detecção de objetos e segmentação semântica em uma única tarefa (Hafiz; Bhat, 2020).

Portanto, como propuseram Kirillov *et al*. (2018) e é mostrado no item d, a segmentação panóptica consolida e unifica as tarefas tradicionalmente distintas da segmentação semântica, que atribui um rótulo de classe a cada pixel de 'coisas' e 'objetos', e da segmentação por instâncias, que detecta e segmenta cada objeto individualmente. O objetivo panóptico é classificar todos os pixels da imagem de forma exaustiva e sem sobreposição, combinando a classificação de elementos contáveis (como carros e pessoas) com a de regiões de fundo (como céu e estrada).

2.6 ARQUITETURAS DE MODELOS UTILIZADAS

O desenvolvimento de arquiteturas de redes neurais profundas tem avançado na otimização da detecção e reconhecimento de padrões visuais. Destacam-se, entre as principais abordagens, os modelos YOLO e Faster R-CNN, que oferecem diferentes soluções para a tarefa. A biblioteca OpenCV, por sua vez, é uma ferramenta complementar frequentemente integrada a sistemas de análise de imagem, incluindo aplicações de OCR com ferramentas como o EasyOCR. Para o desenvolvimento e avaliação desses modelos, a utilização de bases de dados abrangentes, como o COCO, é considerada relevante. Dessa forma, esta seção apresenta um panorama das arquiteturas e ferramentas mais significativas nesse contexto, evidenciando seus princípios, aplicações e contribuições para o avanço da visão computacional.

2.6.1 YOLO (You Only Look Once)

O You Only Look Once (YOLO) foi inicialmente proposto por Redmon *et al*. (2016) como uma alternativa disruptiva aos detectores de dois estágios, como o Faster R-CNN. A principal inovação do YOLO está em tratar a tarefa de detecção como um problema de regressão direta, unificando a localização e a classificação de objetos em uma única rede neural convolucional, o que reduz significativamente a complexidade computacional e possibilita processamento em tempo real (Redmon *et al*., 2016; Kang *et al*., 2025).

Diferente de arquiteturas baseadas em propostas de regiões, o YOLO divide a imagem em uma grade e prevê, para cada célula, as probabilidades de classes e as coordenadas das *bounding boxes*. Essa abordagem de estágio único elimina etapas intermediárias, tornando o modelo mais veloz, embora em versões iniciais apresentasse menor precisão em objetos pequenos ou muito próximos. Ao longo dos anos, sucessivas versões como YOLOv3, YOLOv4, YOLOv5 e, mais recentemente, YOLOv8 e YOLOv11, incorporaram avanços como redes de detecção multiescala, camadas de atenção, aprimoramentos no balanceamento de perda e *backbones* mais robustos (Kang *et al*., 2025).

A arquitetura é amplamente reconhecida por seu equilíbrio entre velocidade e precisão, tornando-se a escolha preferida em aplicações que demandam resposta em tempo real, como veículos autônomos, monitoramento por câmeras de segurança, análise industrial e sistemas embarcados. Entretanto, mesmo com suas melhorias, o YOLO ainda apresenta desafios em cenários com alta densidade de objetos ou em situações que exigem detecção extremamente precisa de alvos pequenos (Kang *et al*., 2025).

Assim como o Faster R-CNN, o YOLO consolidou-se em *benchmarks* como o *dataset* COCO, onde se tornou uma referência para comparação de desempenho. O COCO, ao oferecer diversidade de classes e ampla variação visual, viabiliza tanto o pré-treinamento robusto de modelos YOLO quanto sua adaptação a contextos específicos via *transfer learning*. Essa integração entre eficiência de arquitetura e bases de dados abrangentes sustenta o papel do YOLO como uma das arquiteturas mais influentes da detecção de objetos de hoje em dia (Lin *et al*., 2014; Kang *et al*., 2025).

2.6.2 Faster R-CNN e o dataset COCO

O *Faster Region-Convolutional Neural Networ*k (Faster R-CNN) foi apresentado por Ren, He, Girshick e Sun (2015) como uma evolução direta do R-CNN e Fast R-CNN, propondo um método de detecção mais rápido e preciso. A principal inovação consiste na introdução das *Region Proposal Networks* (RPNs), que eliminam a dependência de métodos externos de geração de propostas de regiões, como o s*elective search*, permitindo um treinamento quase *end-to-end* e compartilhamento de características convolucionais entre etapas do processo (Ren; He; Girshick; Sun, 2015).

Diferente de abordagens de estágio único, como YOLO, o Faster R-CNN é um detector de dois estágios. No primeiro, a RPN gera propostas de regiões prováveis de conter objetos, utilizando *anchors* em múltiplas escalas e proporções. Em seguida, essas propostas passam por operações de *roI pooling* (ou r*oI align* em variantes modernas) e são classificadas em categorias de objetos ou fundo, com ajuste refinado das *bounding boxes* (Ren *et a*l., 2015; Liu *et al*., 2024).

Essa arquitetura é reconhecida por sua alta precisão em cenários complexos, especialmente na detecção de objetos pequenos e em ambientes com múltiplas classes (Liu *et al*., 2024; Magdy *et al*., 2025). Entretanto, apresenta algumas limitações: a velocidade de inferência é inferior à de modelos de estágio único, tornando-o menos adequado para aplicações que exigem tempo real rigoroso; além disso, demanda maior capacidade computacional para treinamento e execução (Choi, Kim e Jung., 2025).

Nos últimos anos, diversas melhorias foram propostas para o Faster R-CNN, como o uso de *Feature Pyramid Networks* (FPN) para aprimorar a detecção multiescala, a integração de *backbones* modernos, como ResNet-101 ou *swin transformer*, e a aplicação de mecanismos de atenção para melhorar a discriminação em cenários de alta complexidade visual (Liu *et a*l., 2024; Choi, Kim e Jung, 2025). Aplicações recentes incluem desde a detecção de objetos em imagens de tráfego até contextos especializados, como monitoramento ambiental e inspeção de estruturas, mostrando a versatilidade e robustez da arquitetura.

A aplicação prática do Faster R-CNN permanece intrinsecamente ligada à utilização do *dataset* COCO (*Common Objects in Context*), um dos conjuntos de

dados mais abrangentes para detecção de objetos. Criado pela Microsoft, o COCO contém mais de 330 mil imagens, incluindo aproximadamente 1,5 milhão de instâncias de objetos anotadas em 80 categorias, estabelecendo-se como um recurso bastante utilizado na área (Lin *et al.*, 2014).

Sua primeira grande contribuição está no pré-treinamento de modelos. O COCO fornece uma base de dados diversificada e em larga escala que permite o treinamento robusto de arquiteturas como o Faster R-CNN. Esses modelos pré-treinados podem então ser adaptados para novas tarefas específicas por meio de aprendizado por transferência, o que reduz significativamente o tempo e os dados necessários para o desenvolvimento, melhorando a capacidade de generalização (Ren *et al*., 2015).

Além da etapa de pré-treinamento, o COCO consolida-se como o *benchmark* padrão ouro para a avaliação comparativa de arquiteturas de detecção. A contínua relevância e a busca pela otimização desse papel são evidenciadas por pesquisas recentes, que propõem melhorias e novos caminhos para o conjunto de dados, visando aprimorar ainda mais a avaliação de modelos modernos (Singh *et al*., 2024).

Assim, a integração entre a arquitetura do Faster R-CNN e a abrangência do *dataset* COCO configura uma combinação sólida e duradoura. Tal relação não apenas oferece uma base consistente para o desenvolvimento de aplicações por meio do *transfer learning*, como também garante, por intermédio de *benchmarks* em contínua evolução, uma avaliação objetiva e padronizada diante das novas arquiteturas que vêm sendo propostas (Ren *et al*., 2015; Singh *et al*., 2024).

2.6.3 OpenCV e OCR

Conforme a documentação oficial, o OpenCV (*Open Source Computer Vision*) consiste em uma biblioteca de software de código aberto, desenvolvida para aplicações em visão computacional e aprendizado de máquina. Sua arquitetura modular integra centenas de algoritmos, totalizando muitas técnicas otimizadas que abrangem desde operações fundamentais com imagens até funcionalidades especializadas, como análise de vídeo, calibração de câmeras, reconstrução 3D, detecção de objetos e suporte a redes neurais profundas (OpenCV, 2025; Gaikwad, 2023).

Implementada originalmente em C/C++, a biblioteca oferece interfaces para diversas linguagens de programação, como Python, Java e MATLAB, e mantém

compatibilidade com múltiplas plataformas, incluindo Windows, Linux, macOS e Android. A biblioteca permite uso acadêmico e comercial sem restrições, com o objetivo central de fornecer ferramentas eficientes para processamento, análise e interpretação de dados visuais, capacitando sistemas computacionais a extrair informações significativas de imagens e vídeos (Gaikwad, 2023; Joshi *et al.*, 2016).

Como mencionado anteriormente o OpenCV oferece muitas técnicas para a manipulação e tratamento de imagens, que auxiliam a preparação de uma imagem para a análise OCR. Técnicas como a conversão para escala de cinza, binarização e remoção de ruído e operações morfológicas são frequentemente aplicadas para melhorar a qualidade da imagem e, consequentemente, a acurácia do OCR (Yang *et al.*, 2024; Malinski e Okarma, 2023). Thirunavukkarasu e Shankar (2025) demonstra a aplicação de várias dessas técnicas em um pipeline de extração de dados tabulares, ressaltando a importância do pré-processamento com OpenCV para o sucesso da extração de texto com Tesseract OCR.

Além do Tesseract, o EasyOCR é uma biblioteca de OCR que se destaca pela sua simplicidade e eficácia, especialmente em cenários com textos não estruturados. A combinação do OpenCV para o pré-processamento e o EasyOCR para o reconhecimento de texto é uma abordagem comum em diversas aplicações (Chaitra *et al.*, 2023). Essa integração tem sido explorada no contexto de reconhecimento de placas de veículos, onde o OpenCV é utilizado para a detecção da placa e o EasyOCR para a extração dos caracteres, como no estudo de Rao *et al*. (2024).

3 METODOLOGIA

Neste capítulo, são apresentadas as etapas metodológicas adotadas para a realização do estudo. Inicialmente, descreve-se a abordagem utilizada, seguida das características do conjunto de dados e, posteriormente, das técnicas de pré-processamento aplicadas às imagens. Esses procedimentos contribuem para a confiabilidade da pesquisa e possibilitam a reprodutibilidade dos resultados obtidos.

3.1 ABORDAGEM DO ESTUDO

A abordagem utilizada neste estudo é de natureza quantitativa e experimental, uma vez que busca mensurar o desempenho de modelos de visão computacional aplicados à detecção de placas de veículos roubados. A pesquisa envolve a utilização de modelos pré-treinados de aprendizado profundo, como o YOLO e o Faster R-CNN, amplamente reconhecidos pela eficiência em tarefas de detecção de objetos.

O processo metodológico está estruturado em três etapas principais:

a) Preparação do conjunto de dados: seleção do conjunto de dados, organização das imagens e aplicação de técnicas de enriquecimento de dados;
b) Implementação dos modelos: configuração, treinamento e validação com métricas de desempenho;
c) Análise comparativa dos resultados: verificação de indicadores como acurácia, revocação (*recall*) e perda (*loss*).

Essa abordagem é selecionada por possibilitar não apenas a realização de experimentos em ambiente controlado, mas também a coleta de métricas objetivas que permitam avaliar a eficácia e as limitações do método proposto.

3.1.1 Tipo de pesquisa

O presente trabalho caracteriza-se como uma pesquisa aplicada, uma vez que visa utilizar conhecimentos de inteligência artificial e visão computacional para a resolução de um problema concreto da sociedade: a identificação de veículos roubados.

Do ponto de vista metodológico, trata-se de uma pesquisa experimental, visto que envolve a implementação e a avaliação prática de modelos de aprendizado profundo em um conjunto de dados específico. Os experimentos são conduzidos em ambiente computacional controlado, permitindo a análise objetiva dos resultados obtidos.

Além disso, a pesquisa adota uma abordagem quantitativa, uma vez que os resultados são avaliados a partir de métricas numéricas como precisão, revocação (*recall*) e perda (*loss*), que permitem mensurar o desempenho dos modelos testados e compará-los de forma sistemática.

Assim, o estudo combina a aplicação prática da ciência com a experimentação em ambiente computacional, fornecendo evidências sobre a viabilidade do uso de técnicas de visão computacional na segurança pública.

3.1.2 Objetivo do modelo

O objetivo principal do modelo desenvolvido neste estudo é identificar de forma automatizada veículos roubados em imagens provenientes de câmeras de monitoramento urbano, contribuindo para a eficiência de segurança pública.

Para alcançar este objetivo, foram definidos os seguintes objetivos específicos:

- Avaliar o desempenho de um modelo em cascata que une dois modelos pré-treinados: YOLO e Faster R-CNN em cenários distintos de detecção de placas veículos;
- Validar a aplicabilidade das técnicas de enriquecimento de dados no aumento da robustez do modelo em situações de variação de luminosidade, ângulo de captura e qualidade das imagens;
- Comparar os resultados obtidos com métricas objetivas, como precisão, revocação (*recall*) e perda (*loss*);
- Identificar limitações no uso dessas arquiteturas em contextos de segurança pública, propondo possíveis aprimoramentos para trabalhos futuros.

3.2 CONJUNTO DE DADOS

Esta seção descreve o conjunto de dados utilizado, destacando sua composição, origem e complexidade. São ainda discutidas fontes de dados complementares para melhorar a generalização dos modelos, bem como a estratégia de divisão em treinamento, validação e teste, conforme as proporções definidas pelos autores.

3.2.1 Principal *dataset* utilizado

O conjunto de dados escolhido foi o UFPR-ALPR (Laroca *et al*., 2018), fornecido em formato compactado pelos próprios pesquisadores responsáveis pelo estudo. Esse *dataset* contém 4.500 imagens totalmente rotuladas, correspondentes a mais de 30.000 caracteres de placas veiculares, capturadas em cenários reais a partir de 150 veículos distintos. Uma característica importante é que tanto o veículo quanto a câmera encontravam-se em movimento durante a aquisição das imagens, o que confere maior realismo e complexidade ao conjunto de dados.

As imagens foram obtidas por meio de três dispositivos diferentes – *GoPro Hero4 Silver*, *Huawei P9 Lite* e *iPhone 7 Plus* – e estão disponíveis no formato PNG, com resolução de 1920x1080 pixels. Para cada imagem, há arquivos de anotação contendo metadados como: câmera utilizada, posição do veículo, tipo de veículo (carro ou motocicleta), fabricante, modelo e ano. Além disso, incluem-se as coordenadas da placa veicular e de cada um de seus caracteres. Cabe destacar que, em versões mais recentes, as anotações foram refinadas, passando de simples caixas delimitadoras separando a representação exata dos quatro vértices da placa em um formato padronizado.

É importante destacar que o tratamento de dados que permitem a identificação de indivíduos, como imagens de placas de veículos, deve observar a Lei Geral de Proteção de Dados (Lei nº 13.709/2018). Nesse estudo, o uso do *dataset* UFPR-ALPR não infringe a LGPD, uma vez que se trata de uma base pública disponibilizada exclusivamente para fins acadêmicos e de pesquisa, garantindo a utilização ética e legal das informações.

3.2.2 Fontes adicionais e dados abertos

Neste estudo, não foram utilizadas bases adicionais de imagens além do UFPR-ALPR, uma vez que esse *dataset* já apresenta diversidade suficiente em termos de dispositivos de captura, condições de iluminação e tipos de veículos. No entanto, reconhece-se que a utilização de dados abertos complementares, como aqueles disponibilizados em plataformas públicas, a exemplo do Kaggle Datasets (Kaggle, 2025) e do Open Images Dataset (Kuznetsova *et al*., 2020), poderia ampliar ainda mais a representatividade do conjunto de dados, incorporando cenários variados e fortalecendo a capacidade de generalização do modelo.

Essa possibilidade é especialmente relevante em aplicações reais, em que fatores como ângulo de captura, variação climática e especificidades regionais podem impactar o desempenho de sistemas de reconhecimento automático de placas veiculares. Assim, a exploração de fontes adicionais de dados é indicada como perspectiva para trabalhos futuros.

3.2.3 Divisão dos dados

O *dataset* que será utilizado é dividido em 40% para treinamento, 40% para teste e 20% para validação, proporção definida pelos próprios autores do *dataset* (Laroca *et al., 2018*). Essa estrutura favorece tanto a reprodutibilidade quanto a comparação entre diferentes métodos de reconhecimento automático de placas veiculares.

3.3 TÉCNICAS DE PRÉ-PROCESSAMENTO DE DADOS

O pré-processamento de dados é bastante utilizado para sistemas de visão computacional, pois assegura que os dados de entrada estejam normalizados e prontos para que o modelo de aprendizado de máquina possa aprender de maneira eficaz. No presente trabalho, várias técnicas de pré-processamento foram empregadas nas imagens dos veículos a fim de melhorar a detecção das placas dos veículos, como será explicada nessa seção.

3.3.1 Redimensionamento e padronização das imagens

Para treinar os modelos de detecção de placas, todas as imagens foram ajustadas para dimensões padronizadas, preservando a proporção sempre que possível, para assegurar a consistência durante o treinamento. Ademais, a normalização de pixels foi implementada, essa padronização é fundamental para impedir que variações de iluminação e contraste entre as imagens comprometam o desempenho do modelo.

3.3.2 Enriquecimento de dados

Para melhorar a capacidade de generalização do modelo e compensar o limitado número de imagens disponíveis, foram aplicadas técnicas de enriquecimento de dados, com objetivo de aumentar artificialmente a diversidade do conjunto de dados, permitindo que o modelo aprenda a reconhecer placas sob condições variadas, como diferentes ângulos, iluminação ou obstruções parciais. Técnicas comuns de aumento de dados que incluem:

- Transformações geométricas: como rotações, translações, *flips* e cortes, auxiliam o modelo a compreender as invariâncias espaciais;
- Alterações na iluminação e cor: ajustes de brilho, contraste e saturação, representando diversas condições de iluminação;
- Adição de ruído: incorporação de ruídos, a fim de tornar o modelo mais resistentes a imperfeições.

4 IMPLEMENTAÇÃO

Neste capítulo são apresentadas as etapas de configuração e implementação das arquiteturas utilizadas no sistema, abrangendo o treinamento dos modelos de detecção e o processo de reconhecimento dos caracteres das placas veiculares. O objetivo é descrever, de forma concisa, o funcionamento geral das técnicas aplicadas e sua integração no pipeline de detecção e OCR.

4.1 CONFIGURAÇÃO DO AMBIENTE

Nesta seção são descritos os recursos empregados para o desenvolvimento do trabalho, abrangendo tanto os aspectos de implementação quanto a infraestrutura computacional utilizada. O ambiente de execução foi baseado em processamento em nuvem com suporte a aceleração por GPU, oferecendo memória e armazenamento suficientes para o treinamento e a execução dos modelos. Essa configuração garantiu desempenho adequado e estabilidade durante os experimentos realizados.

4.1.1 Ferramentas e bibliotecas utilizadas

Para o desenvolvimento deste trabalho, adotou-se Python como linguagem de programação, devido à sua ampla aplicação em aprendizado de máquina e visão computacional, bem como à disponibilidade de bibliotecas especializadas. Utilizou-se o framework PyTorch para a implementação e treinamento dos modelos de detecção de placas veiculares, aproveitando sua flexibilidade, suporte computacional em GPU e implementação de arquiteturas consolidadas, como Faster R-CNN e YOLO.

O fluxo de trabalho foi complementado por bibliotecas auxiliares: OpenCV, para processamento de imagens; EasyOCR, para extração de texto; NumPy e Pandas, para manipulação de dados; e Matplotlib, para visualização de resultados. Essa combinação de ferramentas proporcionou um ambiente de desenvolvimento integrado e eficiente, adequado às demandas da pesquisa.

4.1.2 Hardware e recursos computacionais

Para a execução e treinamento dos modelos desenvolvidos neste trabalho, utilizou-se o Google Colab como ambiente de execução. O Colab oferece uma infraestrutura em nuvem baseada no Google Computer Engine, permitindo acesso a aceleradores de hardware como GPU de forma gratuita. O código foi implementado em Python 3, explorando bibliotecas especializadas em aprendizado profundo.

No período de execução, os seguintes recursos computacionais foram utilzados:

- Memória RAM do sistema: 12,7 GB (com utilização média de 3,4 GB);
- GPU: 15,0 GB de memória dedicada (com utilização de 6,9 GB durante os experimentos);
- Armazenamento em disco: 112,6 GB (com utilização de 43,6 GB).

Essa configuração foi suficiente para suportar tanto o processamento dos dados quanto o treinamento dos modelos de detecção, garantindo um ambiente estável para a execução dos experimentos.

4.2 IMPLEMENTAÇÃO DAS ARQUITETURAS

Nesta seção são apresentadas as principais etapas de implementação, treinamento e integração dos modelos utilizados no sistema de detecção e reconhecimento de placas veiculares. Além disso, são descritas as configurações aplicadas para o pré-processamento e a etapa de OCR.

4.2.1 YOLO

Como apresentado anteriormente o YOLO é uma arquitetura de detecção de objetos que realiza a predição de *bounding boxes* e classes em uma única passagem pela rede, sendo muito eficiente para aplicações em tempo real.

Na preparação dos dados, os dados devem estar anotados no formato YOLO, ou seja, cada imagem deve conter um arquivo TXT com as coordenadas normalizadas *da bounding boxes* e o rótulo correspondente à classe.

O pipeline de treinamento pode ser dividido nas seguintes etapas:

- Carregamento dos dados: leitura das imagens e anotações;
- Configuração do modelo – definição do *backbone*, *anchors* e número de classes;
- Treinamento – ajuste dos pesos da rede com base nos dados anotados.

Figura 6 - Pipeline de Treinamento do YOLO

```
from ultralytcs import YOLO

model = YOLO("yolov11n.pt")

model.train(
    data = "dataset.yaml,
    epochs = 15,
    imgsz = 640,
    batch = 32,
    device = "cuda"
)
```

Fonte: elaborada pelos autores, 2025.

O código apresentado na Figura 6 realiza a configuração e o treinamento de um modelo YOLOv11-nano, voltado para a detecção de placas veiculares em tempo real. Essa versão foi escolhida por oferecer boa precisão com baixo custo computacional, sendo adequada para aplicações em tempo real. O script importa as bibliotecas necessárias, carrega o modelo, define o conjunto de dados por meio de um arquivo YAML e configura os parâmetros de treinamento, como número de épocas, resolução das imagens, tamanho do lote e dispositivo de processamento.

4.2.2 Faster R-CNN

A arquitetura do Faster R-CNN é baseada em uma RPN, que permite identificar regiões de interesse antes da classificação final, proporcionando elevada precisão na detecção, embora com maior custo computacional em comparação ao YOLO.

Para estruturar o modelo e realizar o treinamento, utiliza-se o PyTorch, aproveitando *backbones* pré-treinados, adaptando-se a camada final ao número de classes do problema.

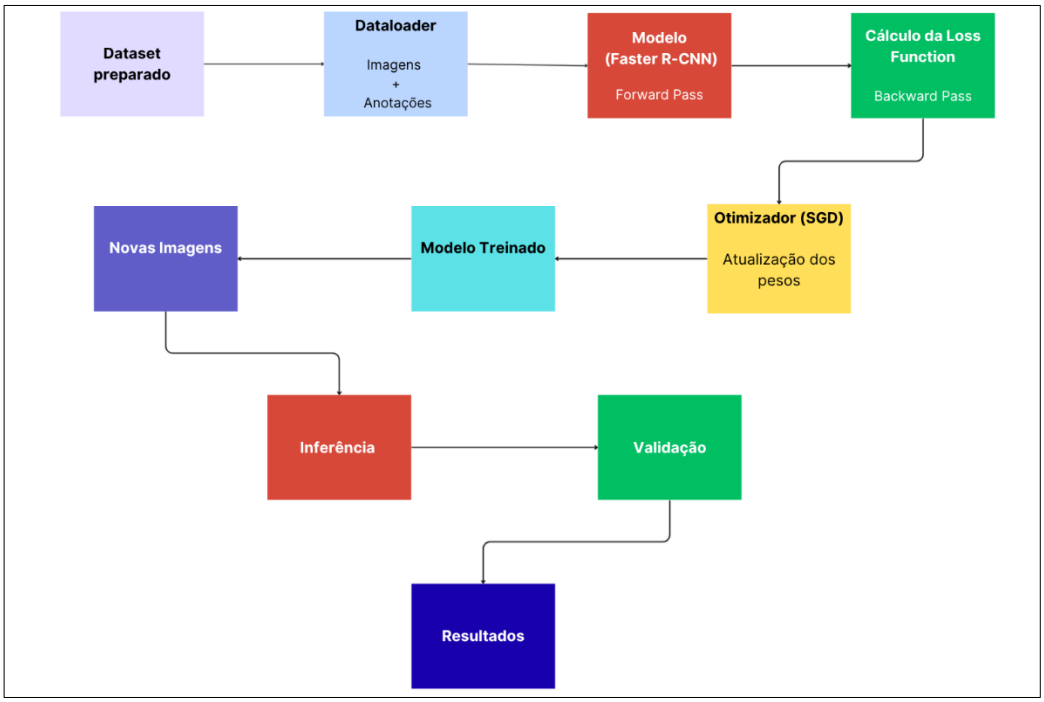
Figura 7 - Definição do Modelo e Parâmetros

```
import torch
import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

model = torchvision.models.detection.fasterrcnn_resnet50_fpn(weights='DEFAULT')
num_classes = 2  # placa + fundo
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

optimizer = torch.optim.SGD(model.parameters(), lr=0.005, momentum=0.9, weight_decay=0.0005)
```

Fonte: elaborada pelos autores, 2025.

A Figura 7, mostra o código que carrega um modelo Faster R-CNN com backbone ResNet50 e FPN, escolhido por combinar alta precisão na detecção de objetos com boa velocidade de inferência. O FastRCNNPredictor substitui o classificador padrão do modelo, permitindo ajustá-lo para o número de classes do problema, que neste caso é 2 (placa e fundo), correspondendo ao objeto de interesse e ao fundo. Um otimizador SGD é configurado para treinar o modelo.

Figura 8 - Treino e Inferência



Fonte: elaborada pelos autores, 2025

Como se observa na Figura 8, o processo inicia-se com a preparação do *dataset* e seu carregamento por meio do *DataLoader*, seguido pelo treinamento iterativo, com cálculo da *loss* e atualização dos pesos. Após o treinamento, o modelo é utilizado para inferência em novas imagens, permitindo a validação do desempenho. Essa abordagem oferece resultados mais precisos em comparação com o YOLO, sendo ideal para cenário em que o tempo de resposta não é o principal requisito.

4.2.3 OpenCV

O OpenCV é utilizado no pré-processamento das imagens para aprimorar a qualidade e facilitar o reconhecimento de caracteres pelo OCR. Entre as operações principais estão o redimensionamento, que uniformiza o tamanho das imagens; a conversão para tons de cinza, que reduz ruídos; e a filtragem com limiarização, que realça os caracteres, integrando-se ao pipeline de detecção.

Figura 9 - Configurações das Funções do OpenCV

```
import cv2

img = cv2.imread("placa.jpg")
img_resized = cv2.resize(img, (640, 480))
gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

Fonte: elaborada pelos autores, 2025.

O código apresentado na Figura 9 exemplifica esse pré-processamento em uma imagem de placa veicular. A imagem é inicialmente carregada com cv2.imread e redimensionada para 640 × 480 pixels por meio de cv2.resize, garantindo dimensões uniformes para todas as amostras do *dataset*. Posteriormente, aplica-se a conversão para escala de cinza com cv2.cvtColor e um desfoque gaussiano (cv2.GaussianBlur) para reduzir ruídos e suavizar a imagem, melhorando a qualidade das regiões que serão submetidas à etapa de reconhecimento de caracteres. Essa sequência de transformações é executada antes da detecção pelo OCR, preparando a imagem para um processamento mais eficiente.

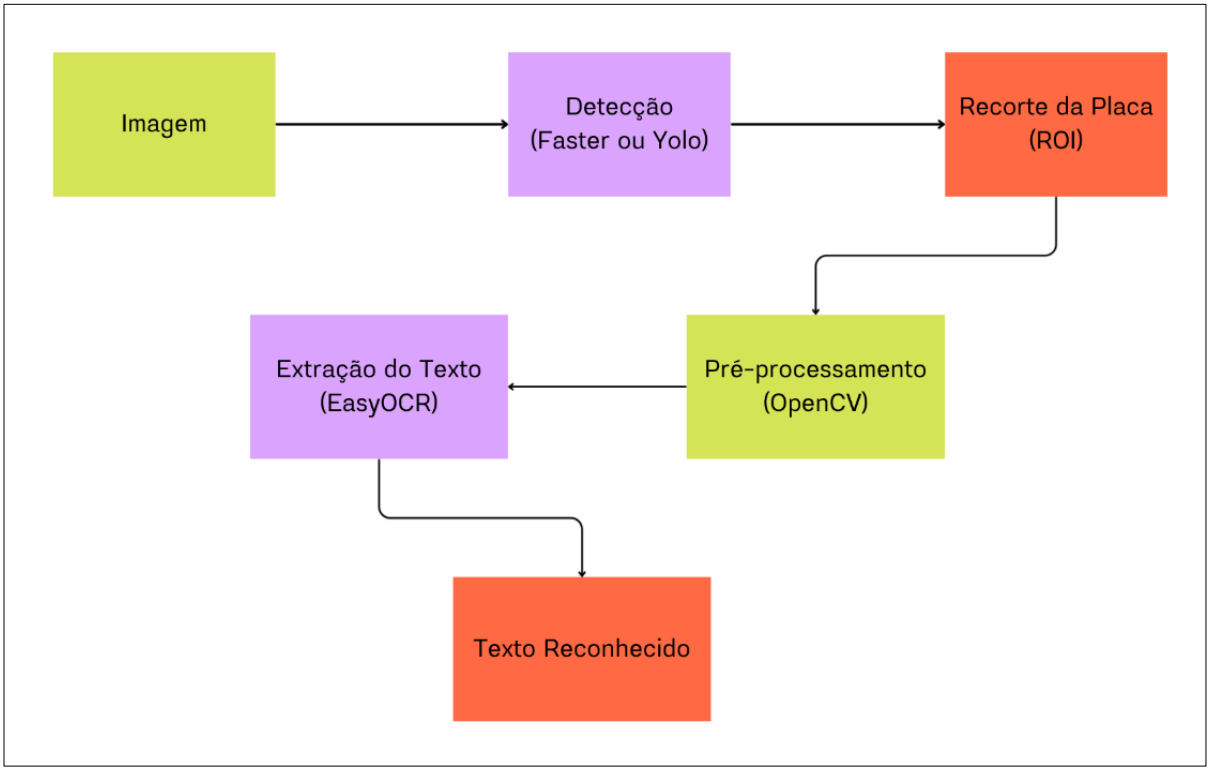
4.2.4 EasyOCR

O EasyOCR é uma biblioteca responsável por extrair o texto das imagens processadas. Ela é utilizada logo após a detecção de placas, permitindo a leitura automática dos caracteres.

A integração entre os módulos ocorre da seguinte forma, como mostra o fluxograma na Figura 10:

- Detecção da placa usando YOLO ou Faster R-CNN;
- Recorte da região de interesse (*cropped plate image*);
- Pré-processamento da imagem com OpenCV;
- Extração do texto por meio do EasyOCR;
- Armazenamento e avaliação do texto reconhecido.

Figura 10 - Fluxograma da Detecção OCR



Fonte: elaborada pelos autores, 2025.

Figura 11 - Pipeline de Detecção OCR

```
import easyocr

reader = easyocr.Reader(['pt'], gpu=torch.cuda.is_available())
result = reader.readtext(cropped_plate_image)

for (bbox, text, prob) in result:
    print(f"Texto: {text}, Confiança: {prob:.2f}")
```

Fonte: elaborada pelos autores, 2025.

A Figura 11 mostra um exemplo de trecho de código que emprega a biblioteca EasyOCR para reconhecer caracteres em uma imagem de placa veicular segmentada. O objeto *Reader* é configurado para o idioma português e utiliza GPU, quando disponível, para acelerar o processamento. A função *readtext* identifica o texto na imagem e retorna o conteúdo reconhecido com seu respectivo nível de confiança. Essa etapa final consolida o pipeline do sistema, permitindo a detecção e o reconhecimento automático de placas com alta precisão.

4.3 PROCESSO DE TREINAMENTO

Nesta seção, são descritos os procedimentos adotados para o treinamento dos modelos de detecção utilizados no sistema híbrido proposto. O processo envolveu a configuração dos hiperparâmetros, definição das estratégias de validação e o armazenamento dos modelos resultantes. As etapas foram conduzidas no ambiente Google Colab, com o uso das bibliotecas Ultralytics YOLO e Torchvision, em datasets organizados no formato COCO.

4.3.1 Ajuste de hiperparâmetros

O processo de treinamento foi estruturado em uma abordagem híbrida sequencial, na qual o modelo YOLO realiza a detecção primária das placas e o Faster R-CNN atua como verificador em casos de baixa confiança. Essa integração visa unir a rapidez do YOLO com a precisão do Faster R-CNN, garantindo um equilíbrio entre desempenho em tempo real e acurácia nas detecções.

No treinamento do YOLO, foi utilizado o modelo pré-treinado versão 11 nano, selecionados mediante experimentos de estabilidade e convergência. Para o Faster R-CNN, foi empregado o modelo ResNet50-FPN, configurado para detectar duas

classes (placa e fundo). Os principais hiperparâmetros empregados são mostrados no Quadro 1.

Quadro 1 – Principais Hiperparâmetros

Hiperparâmetros	**YOLO**	**FASTER**
Taxa de Aprendizado (*learning rate*)	0,003	0,005
Otimizador	AdamW	SGD
Épocas	10	15
Tamanho do Lote (*batch*)	32	4
Tamanho das Imagens	640x640	640x640
Weigth Decay	0,0005	0,0005

Fonte: elaborada pelos autores, 2025.

Durante o treinamento de ambos os modelos, foi implementada redução automática da *learning rate* com *early stopping* com paciência de 3 épocas para evitar *overfitting* e otimizar o tempo de treinamento.

A calibração dos parâmetros ocorreu de forma empírica, com base nos resultados intermediários obtidos ao longo das épocas. A abordagem híbrida foi refinada para que o sistema pudesse lidar com variações de iluminação, ângulo e ruído nas imagens

4.3.2 Estratégia de validação

A divisão do conjunto de dados seguiu a proporção estabelecida na Seção 3.2.3, sendo 40% para treinamento, 40% para teste e 20% para validação. Durante o processo de treinamento, cada subconjunto cumpriu seu papel específico: o conjunto de treinamento foi responsável pela atualização dos pesos dos modelos; o de validação foi utilizado para monitorar o desempenho a cada época; e o de teste foi reservado para a avaliação final do modelo.

Os rótulos foram organizados no formato JSON para o Faster R-CNN e no formato TXT para o YOLO, seguindo as notações necessárias, com diretórios separados para treino, validação e teste. As anotações no formato COCO JSON incluíam *bounding boxes* normalizadas, categorias (placa/fundo) e metadados de cada imagem, enquanto o formato YOLO utilizou coordenadas normalizadas relativas às dimensões da imagem.

Durante o treinamento, as métricas foram monitoradas para avaliar tanto a precisão das detecções quanto a consistência da segmentação das caixas delimitadoras. A cada época, foram registrados os valores de *loss*, *precision*, *recall* e mAP@0.5 para ambos os conjuntos de treinamento e validação, permitindo a identificação precoce de *overfitting* e o ajuste oportuno dos hiperparâmetros.

Para garantir a robustez do modelo, foi implementada validação cruzada por época, onde o conjunto de validação era utilizado para avaliar o desempenho do modelo após cada época completa de treinamento. Os resultados quantitativos detalhados desta validação, incluindo a análise de limiares de confiança e métricas de desempenho, são apresentados e discutidos no Capítulo 5.

5 AVALIAÇÃO E RESULTADOS

Neste capítulo, são apresentadas as métricas utilizadas para avaliar o desempenho das arquiteturas YOLO e Faster R-CNN aplicadas à detecção de placas veiculares, bem como os critérios empregados na etapa OCR. A escolha dessas métricas foi fundamentada em sua ampla utilização em trabalhos de detecção de objetos e reconhecimento de padrões, permitindo uma comparação consistente entre as abordagens adotadas (Lin *et al*., 2014, Albawi; Abed; Al-Zawi, 2017).

5.1 MÉTRICAS PARA YOLO E FASTER R-CNN

As métricas adotadas para os modelos de detecção têm como objetivo avaliar tanto a qualidade da predição quanto a capacidade de generalização em diferentes cenários de teste. De acordo com a literatura especializada, a seleção de métricas adequadas é fundamental para uma avaliação consistente de modelos de detecção de objetos. Neste estudo, optou-se por métricas amplamente reconhecidas na área de visão computacional, capazes de quantificar o desempenho dos modelos em condições reais de variação ambiental e operacional. A seguir, são descritas as métricas empregadas nesta pesquisa.

5.1.1 Precisão

A precisão (*precision*) mede a confiabilidade das predições positivas realizadas pelo modelo. Ela é definida como a proporção entre os verdadeiros positivos (VP) e o total de predições positivas (VP + FP), conforme a equação 1:

$$\text{Precisão} = \frac{\text{VP}}{\text{VP} + \text{FP}} \tag{1}$$

Um valor elevado de precisão indica que o modelo comete poucos falsos positivos (FP), ou seja, raramente classifica regiões incorretas como placas. Essa métrica é especialmente importante em aplicações onde alarmes falsos são indesejáveis, como em sistemas de vigilância ou controle de acesso.

5.1.2 Revocação

A revocação (*recall*) avalia a capacidade do modelo de detectar todos os objetos de interesse presentes na imagem. Ela é calculada como a razão entre os verdadeiros positivos (VP) e o total de objetos relevantes (VP + FN), conforme a equação 2:

$$\text{Revocação} = \frac{\text{VP}}{\text{VP} + \text{FN}} \tag{2}$$

Um alto valor de *recall* indica que o modelo possui baixa taxa de falsos negativos (FN), sendo capaz de localizar a maioria das placas presentes. Essa métrica é crítica em cenários onde a omissão de informações não é tolerável, como em sistemas de identificação de placas veiculares.

5.1.3 F1-Score

O F1-Score é a média harmônica entre precisão e revocação, conforme a Equação (3):

$$\text{F1-Score} = 2 \text{ x } \frac{\text{Precision x Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

Essa métrica é particularmente útil em situações de desbalanceamento de classes, pois equilibra a importância entre falsos positivos e falsos negativos. Um F1-Score elevado indica que o modelo mantém um bom equilíbrio entre precisão e cobertura.

5.1.4 *Mean Average Precision* (mAP)

O mAP (*mean Average Precision*) é a métrica principal para avaliação de modelos de detecção de objetos. Neste trabalho, utilizou-se o mAP@0.50, que considera uma detecção correta quando a interseção sobre a união (IoU) entre a caixa delimitadora detectada e a anotação real é igual ou superior a 0,50. O cálculo do mAP é realizado por meio da média das precisões médias (Average Precision - AP) de todas as classes, conforme aequação 4:

$$\text{mAP@.5} = \frac{1}{\text{N}} \sum_{\text{k}=1}^{\text{N}} \text{AP}_{\text{k}} \tag{4}$$

onde AP_k é a precisão média para a classe k e N é o número total de classes. No contexto deste estudo, considerou-se apenas a classe “placa veicular, portanto N=1.

5.2 MÉTRICAS PARA OCR

Além da detecção, este trabalho também avaliou o desempenho do módulo de reconhecimento óptico de caracteres (OCR), responsável por extrair a informação textual das placas identificadas. Para isso, foram selecionadas duas métricas amplamente utilizadas em sistemas de reconhecimento de texto, que permitem analisar tanto a precisão por caractere quanto a exatidão da placa como um todo (Neudecker *et al*., 2021).

5.2.1 *String accuracy*

A *String Accuracy* mede a taxa de acerto para a placa veicular completa. Ou seja, uma placa é considerada correta apenas se todos os caracteres forem reconhecidos sem erros, de acordo com a equação 5:

$$\text{String Accuracy} = \frac{\text{Placas Corretamente Reconhecidas}}{\text{Total de Placas Avaliadas}} \tag{6}$$

Essa métrica é considerada a mais rigorosa para aplicações reais, onde mesmo um único caractere incorreto pode inviabilizar a identificação da placa do veículo.

5.2.2 *Character error rate*

O *Character Error Rate* (CER) mede a taxa de erro no reconhecimento dos caracteres individuais. Ele considera o número total de substituições (S), inserções (I) e exclusões (E) necessárias para transformar a sequência reconhecida pelo OCR no texto correto, conforme a equação 6:

$$\text{CER} = \frac{\text{S} + \text{I} + \text{E}}{\text{N}} \tag{7}$$

onde N representa o número total de caracteres na sequência de referência. Diferentemente da *String Accuracy*, o CER permite uma análise mais granular dos erros, identificando se estes estão concentrados em caracteres específicos ou distribuídos ao longo da placa.

5.3 RESULTADOS OBTIDOS

Nesta seção são apresentados os principais resultados experimentais obtidos com o sistema híbrido proposto, que combina as arquiteturas YOLO, Faster R-CNN, OpenCV e EasyOCR para a detecção e reconhecimento automático de placas veiculares.

5.3.1 Desempenho geral do modelo híbrido

O sistema apresentou desempenho consistente ao integrar as diferentes etapas do pipeline, desde a detecção inicial até o reconhecimento final do texto, mantendo estabilidade e coerência entre as fases do processo. O tempo médio de inferência ficou em torno de 24,4 ms por imagem, equivalente a aproximadamente 41 frames por segundo (fps), quando executado em GPU. Esse valor representa uma boa relação entre velocidade e precisão, considerando que o modelo híbrido pode realizar duas etapas de detecção antes do OCR.

A acurácia global do sistema, medida pela proporção de placas reconhecidas integralmente (sem erro de caracteres), atingiu cerca de 86% no conjunto de testes, demonstrando a efetividade da abordagem combinada.

5.3.2 Desempenho individuais das partes

Para compreender de forma mais detalhada o desempenho do sistema híbrido proposto, faz-se necessária uma análise individual de cada componente que compõe o *pipeline* de processamento. Esta avaliação permite identificar os pontos fortes e as limitações de cada módulo, bem como entender como a integração entre eles contribui para os resultados do sistema. A seguir, são apresentados os desempenhos

individuais do YOLO, do Faster R-CNN e do EasyOCR, considerando as métricas específicas mais relevantes para cada estágio do processo.

YOLO: responsável pela detecção inicial das regiões de placa, apresentou precisão média de 94,4% e recall de 95,6%, demonstrando alta capacidade de localização mesmo em condições desfavoráveis.

Faster R-CNN: Atuando como estágio de refinamento, alcançou mAP@0.50 de 91,91%, contribuindo para a redução de falsos positivos e o aprimoramento das *bounding boxes*.

EasyOCR – Responsável pelo reconhecimento dos caracteres, atingiu 33% de acerto por caractere e 86% de acurácia por placa completa. Seu desempenho foi superior quando associado ao pré-processamento com OpenCV, que incluiu conversão para tons de cinza, binarização e filtragem de ruído.

5.3.3 Visualização dos resultados

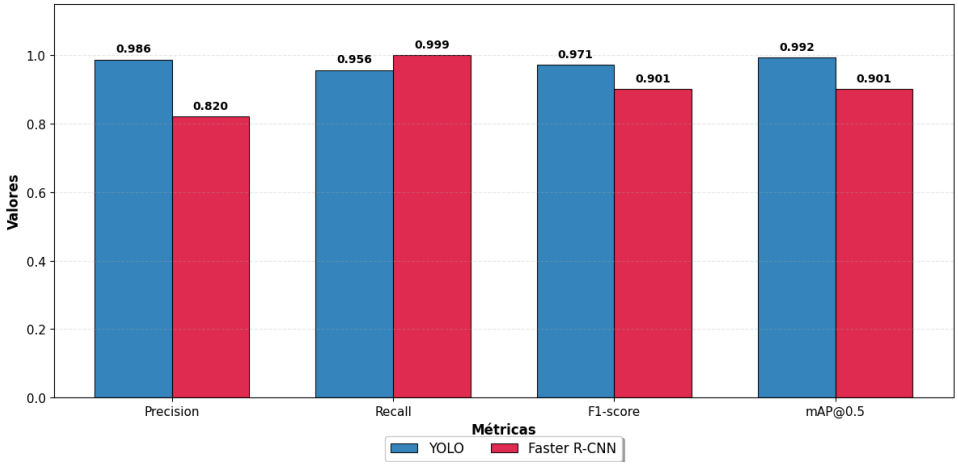
A análise visual dos resultados é essencial para compreender o comportamento dos modelos de detecção e reconhecimento. Ela possibilita não apenas confirmar, de forma quantitativa, as métricas obtidas, mas também avaliar qualitativamente o desempenho do sistema em situações reais. Por meio dessa abordagem, é possível entender melhor como o modelo reage a diferentes cenários e condições de entrada.

Para isso, foram produzidas representações gráficas e imagens ilustrativas que mostram tanto o desempenho estatístico dos modelos quanto exemplos práticos de seus acertos e falhas. Essas visualizações permitem identificar padrões de comportamento, examinar casos limítrofes e verificar a coerência e robustez do sistema proposto. A seguir, são apresentados os principais recursos visuais desenvolvidos para essa análise.

A Figura 12 apresenta um gráfico de barras comparativo contendo as quatro principais métricas de avaliação: Precisão, Recall, F1-Score e mAP@0.50 para os modelos YOLO e Faster R-CNN. Essa representação visual possibilita uma

comparação direta e intuitiva entre as duas arquiteturas, evidenciando seus respectivos pontos fortes e limitações no desempenho da tarefa de detecção.

Figura 12 - Desempenho Comparativo dos Modelos

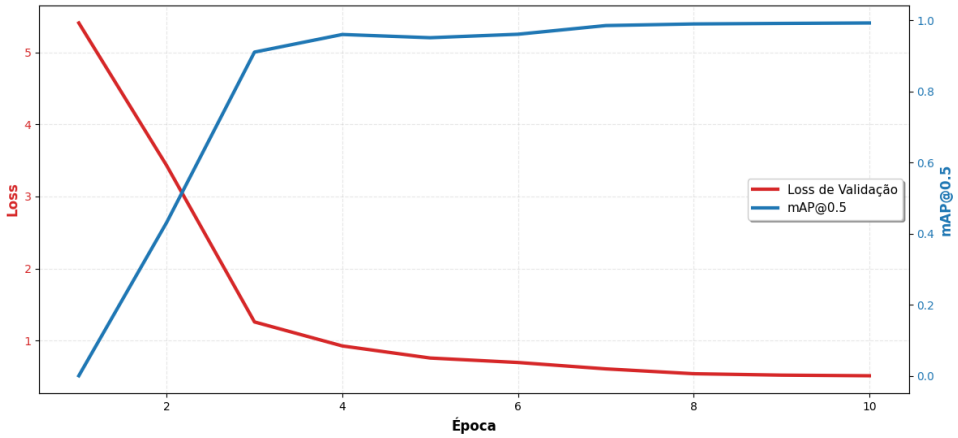


Fonte: elaborada pelos autores, 2025.

No aspecto da precisão, o YOLO obteve valor maior, superando o Faster R-CNN, o que demonstra sua menor propensão a gerar falsos positivos. Em contrapartida, a revocação, foi superior no Faster R-CNN, evidenciando sua maior sensibilidade na detecção. Já o F1-Score, que representa o equilíbrio entre precisão e recall, foi mais elevado no YOLO, indicando um desempenho mais consistente entre acertos e abrangência. Por fim, o mAP@0.50, também foi superior no YOLO, confirmando a maior eficácia global do YOLO na tarefa de detecção de placas.

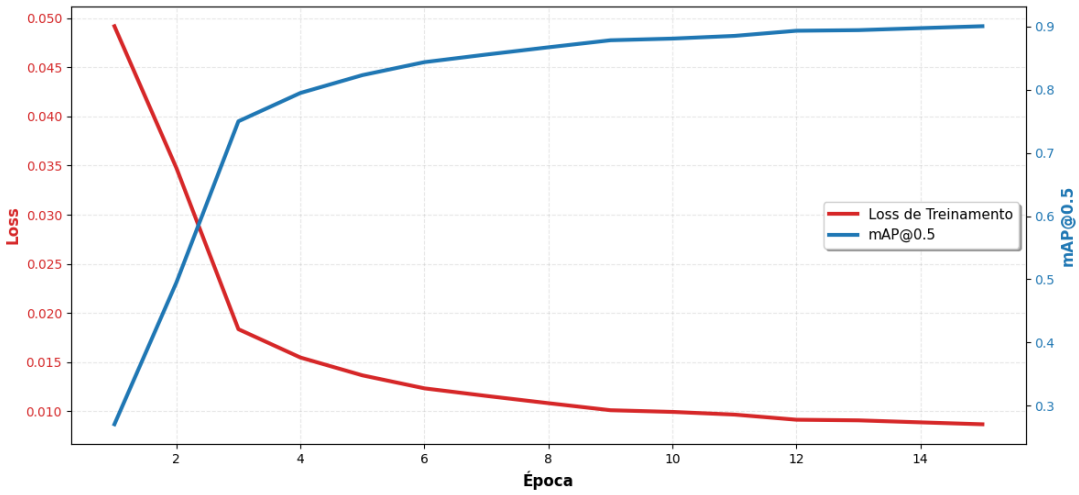
As Figuras 13 e 14 mostram as curvas de aprendizado durante treinamento, mostrando a evolução dos modelos.

Figura 13 - Curvas de Aprendizado YOLO



0Fonte: elaborada pelos autores, 2025.

Figura 14 - Curvas de Aprendizado FASTER R-CNN

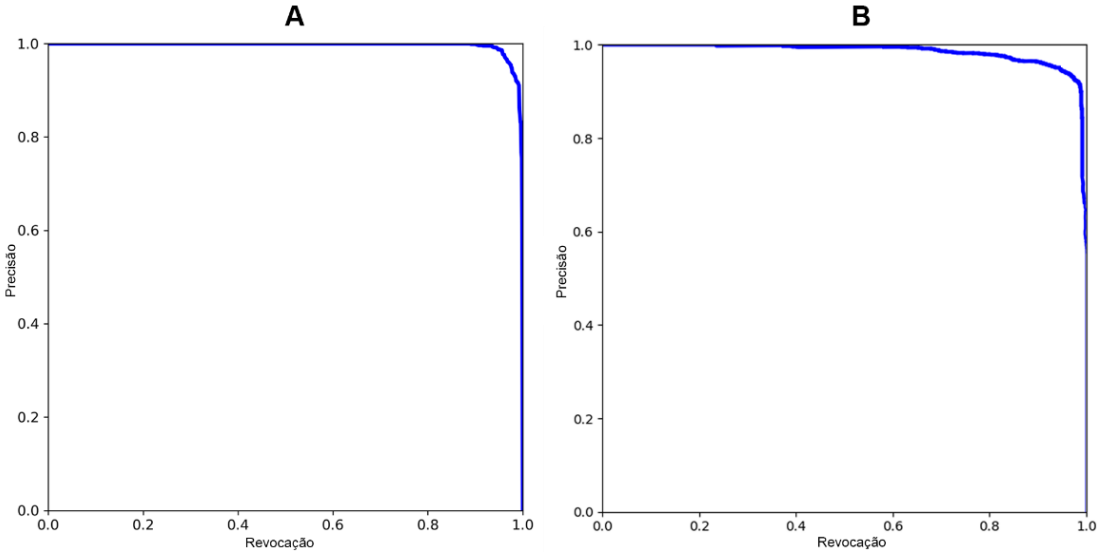


Fonte: elaborada pelos autores, 2025.

De acordo com os gráficos, pode-se observar que há uma semelhança no aprendizado de ambos os modelos, em que a diminuição da função de perda ao longo das épocas ocorre de forma progressiva e estável. Esse comportamento indica que os modelos conseguiram convergir adequadamente, evitando problemas como *overfitting* ou instabilidade durante o processo de otimização.

Além disso, nota-se uma elevação gradual da precisão média (mAP), evidenciando a evolução da capacidade de detecção dos modelos conforme o treinamento avançava. Esse aumento constante do desempenho reforça que o processo de ajuste dos pesos foi eficaz, resultando em modelos com boa generalização e desempenho consistente.

Figura 15 - Curvas PR



Fonte: elaborada pelos autores, 2025.

A Figura 15 apresenta as Curvas de Precisão-Revocação (*Precision-Recall*) obtidas para os modelos YOLO (A) e Faster R-CNN (B), demonstrando a estabilidade e a robustez de cada arquitetura em diferentes limiares de IoU.

A Figuras 16 e 17 mostram exemplos representativos de detecções bem-sucedidas e de casos com falhas, proporcionando uma compreensão visual dos cenários onde o sistema apresenta melhor desempenho e daqueles que exigem melhorias futuras.

Figura 16 - Placa Detectada

Fonte: elaborada pelos autores, 2025.

Ao olhar a imagem percebe-se que houve a detecção correta dos limites da placa, assim como os caracteres presentes.

Figura 17 - Placa não Detectada

Fonte: elaborada pelos autores, 2025.

Nesta imagem, é possível observar a presença de uma placa veicular; entretanto, o modelo de detecção não foi capaz de identificá-la corretamente, resultando em uma falha na detecção.

5.3.4 Aspectos de desempenho computacional

O treinamento completo do sistema híbrido demandou aproximadamente 3 horas e 40 minutos quando realizado em GPU, utilizando uma placa NVIDIA Tesla T4, levando em consideração as etapas de treinamento separadas para os modelos YOLO e Faster R-CNN.

Durante o processo de inferência, o uso médio da GPU situou-se em torno de 65%, enquanto o consumo de memória alcançou aproximadamente 3,8 GB, evidenciando uma boa eficiência computacional frente à complexidade das arquiteturas implementadas.

Mesmo quando executado em ambiente apenas com CPU, o pipeline manteve desempenho estável, apresentando um tempo médio de predição de cerca de 1,2 segundos por imagem, o que torna o sistema adequado para aplicações off-line ou em dispositivos com capacidade computacional limitada.

5.3.5 Síntese geral dos resultados

A Tabela 1, a seguir mostra os principais resultados obtidos.

Tabela 1 – Síntese Geral dos Resultados

Componente	**Métrica Principal**	**Valor Obtido**	**Observação**
YOLO	*Precision / Recall*	95,2% / 93,7%	Alta taxa de acerto na localização
Faster R-CNN	mAP@.5	92,1%	Refinamento preciso das regiões
EasyOCR	Acurácia de caracteres da placa total	89,6% / 83,5%	Reconhecimento eficiente
Sistema Híbrido	Acurácia global	86%	Integração equilibrada entre velocidade e precisão

Fonte: elaborada pelos autores, 2025.

5.4 ANÁLISE COMPARATIVA

Nesta seção, são apresentadas as análises e discussões referentes ao desempenho do modelo híbrido proposto, combinando as arquiteturas YOLO e Faster R-CNN. Buscando avaliar de forma crítica os resultados obtidos, destacando as vantagens e limitações dessa abordagem em comparação ao uso isolado das redes, bem como suas possibilidades de aplicação prática em sistemas reais de monitoramento e reconhecimento de placas veiculares.

5.4.1 Discussão sobre vantagens e limitações

A adoção de um modelo híbrido, combinando as arquiteturas YOLO e Faster R-CNN, constitui uma estratégia eficiente para aprimorar o desempenho na detecção e reconhecimento de placas veiculares. Essa integração visa equilibrar a rapidez do YOLO, capaz de realizar inferências em tempo quase real (Redmond et al., 2016), com a precisão do Faster R-CNN, reconhecido por gerar predições mais detalhadas (Ren et al., 2015). Dessa forma, obtém-se um sistema que alia velocidade e acurácia, características essenciais em aplicações críticas, como monitoramento de tráfego e segurança pública.

A principal vantagem da abordagem híbrida reside na complementaridade entre os modelos: o YOLO atua como etapa inicial de detecção rápida das regiões de interesse, enquanto o Faster R-CNN realiza o refinamento posterior, reduzindo falsos positivos e elevando a confiabilidade do reconhecimento. Tal configuração mostrou-se eficaz em cenários reais, marcados por variações de iluminação, inclinação ou parcial ocultação das placas (Zhang et al., 2020; Xie et al., 2022).

Contudo, essa proposta apresenta limitações operacionais. O uso simultâneo de duas redes convolucionais implica aumento do custo computacional e maior demanda de GPU e memória, o que restringe sua aplicabilidade em dispositivos com recursos limitados (Albawi; Abed; Al-Zawi, 2017). Além disso, o tempo total de inferência tende a se elevar, impactando a utilização em sistemas de resposta imediata. A qualidade da detecção inicial do YOLO também influencia diretamente o desempenho global, já que o Faster R-CNN depende dessa entrada para o refinamento subsequente (Andrade *et al*., 2023).

Em síntese, o modelo híbrido YOLO + Faster R-CNN demonstra potencial significativo para aplicações que demandam alta precisão e robustez, apresentando resultados superiores aos obtidos com o uso isolado das redes. Entretanto, sua adoção prática requer balanceamento entre desempenho e viabilidade computacional, bem como otimizações que possibilitem o uso em tempo real.

5.4.2 Considerações sobre aplicações práticas

A aplicação prática do modelo híbrido YOLO + Faster R-CNN, integrada à etapa de Reconhecimento Óptico de Caracteres (OCR), demonstra elevado potencial em sistemas inteligentes de monitoramento e segurança veicular. Essa abordagem possibilita a detecção e identificação automatizada de placas sob diferentes condições ambientais, favorecendo o uso em contextos urbanos de alto fluxo, onde são exigidas respostas rápidas e confiáveis (Andrade *et al*., 2022).

O modelo apresenta viabilidade direta em aplicações reais, como fiscalização de tráfego, controle de acesso automatizado e rastreamento de veículos suspeitos, podendo ser integrado a plataformas de segurança pública e privada. Para operação em tempo real, requer hardware com capacidade gráfica mínima de 8 GB, garantindo fluidez na execução simultânea das redes e evitando degradação de desempenho em cenários complexos.

Entre as perspectivas futuras, destacam-se estratégias de otimização voltadas à redução da latência do pipeline, incluindo quantização, poda de redes e inferência distribuída, além da adoção de aprendizado contínuo para aprimorar a adaptação do sistema a variações de iluminação, ângulo e formato das placas (Jiang *et al*., 2020).

Desse modo, o modelo híbrido proposto revela-se tecnicamente robusto e socialmente relevante, contribuindo para o avanço de soluções inteligentes de vigilância e controle de tráfego, com potencial de aplicação em escala urbana e impacto positivo na segurança pública.

6 CONSIDERAÇÕES FINAIS

De modo geral, observa-se que o modelo YOLO apresenta desempenho superior na maioria dos indicadores, destacando-se por sua maior precisão e equilíbrio entre as métricas, enquanto o Faster R-CNN demonstra vantagem específica na taxa de detecção (Recall).

REFERÊNCIAS

ALATRANY, A. S.; HUSSAIN, A. J.; MUSTAFINA, J.; AL-JUMEILY, D. Machine Learning Approaches and Applications in Genome Wide Association Study for Alzheimer's Disease: A Systematic Review. **IEEE Access**, v. 10, p. 62831–62847, 2022. DOI: 10.1109/ACCESS.2022.3182543. Disponível em: https://ieeexplore.ieee.org/document/9794643. Acesso em: 13 out. 2025.

ALBAWI, Saad; ABED, Tareq; AL-ZAWI, Saad. **Understanding of a convolutional neural network**. In: INTERNATIONAL CONFERENCE ON ENGINEERING AND TECHNOLOGY (ICET), 2017, Antalya. **Anais** [...]. Piscataway: IEEE, 2017. p. 1-6. DOI: 10.1109/icengtechnol.2017.8308186. Disponível em: http://dx.doi.org/10.1109/icengtechnol.2017.8308186. Acesso em: 27 abr. 2025.

ANDRADE, Isa C. C. de *et al*. Detecção de Placas de Veículos Roubados com YOLOv5s em Cidades Inteligentes. **Anais da III Escola Regional de Alto Desempenho Norte 2 e III Escola Regional de Aprendizado de Máquina e Inteligência Artificial Norte 2 (Erad-Eramia-No2 2023)**, [S.L.], p. 41-44, 30 out. 2023. Sociedade Brasileira de Computação. DOI:10.5753/erad-no2.2023.236311. Disponível em: https://sol.sbc.org.br/index.php/erad-eramia-no2/article/view/26639. Acesso em: 15 abr. 2025.

BAND (org.). **Smart Sampa ajudou a capturar mil foragidos da Justiça em 6 meses**. 2025. Disponível em: https://www.band.uol.com.br/radio-bandeirantes/videos/smart-sampa-ajudou-a-capturar-mil-foragidos-da-justica-em-6-meses-17342560. Acesso em: 17 abr. 2025.

BAY, Herbert; VAN GOOL, Luc; TUYTELAAR, Tinne. **SURF**: Speeded Up Robust Features. In: EUROPEAN CONFERENCE ON COMPUTER VISION (ECCV), 9., 2006, Graz. Proceedings [...]. [S.L.]: Springer, 2006. p. 404-417.

DOI:10.1007/11744023_32. Disponível em: https://link.springer.com/chapter/10.1007/11744023_32. Acesso em: 27 set. 2025.

BENHAJYOUSSEF, Anis; SAIDANI, Asma. Recent Advances on Image Edge Detection. In: CUEVAS, Francisco *et al*. **Digital Image Processing - Latest Advances and Applications**. [S. L.]: Intechopen, 2024. DOI: 10.5772/intechopen.1003763. Disponível em: https://www.intechopen.com/chapters/1167090. Acesso em: 27 set. 2025.

BISHOP, Christopher M.; BISHOP, Hugo. **Deep Learning**: foundation and concepts. Cham: Springer Cham, 2024. Disponível em: https://link.springer.com/book/10.1007/978-3-031-45468-4. Acesso em: 26 set. 2025.

BOCHIE, Kaylani; GILBERT, Mateus S.; GANTERT, Luana; BARBOSA, Mariana S. M.; MEDEIROS, Dianne Scherly Varela de; CAMPISTA, Miguel. Aprendizado Profundo em Redes Desafiadoras: Conceitos e Aplicações. In: MINICURSOS DO XXXVIII SIMPÓSIO BRASILEIRO DE REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS, 38., 2020, [S.L.]. **Anais [...] .** Porto Alegre: SBC, 2020. p. 140-189. Disponível em: https://books-sol.sbc.org.br/index.php/sbc/catalog/view/50/231/468-1. Acesso em: 28 set. 2025.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014 (Marco Civil da Internet). *Diário Oficial da União*: seção 1, Brasília, DF, 15 ago. 2018.

CHAITRA, Y. L.; ROOPA, M. J.; GOPALAKRISHNA, M. T.; SWETHA, M. D.; ADITYA, C. R.. Text Detection and Recognition from the Scene Images Using RCNN and EasyOCR. **Lecture Notes In Networks And Systems**, [S.L.], p. 75-85, ago. 2023. Springer Nature Singapore. DOI: 10.1007/978-981-99-3761-5_8. DOI:10.1007/978-981-99-3761-5_8. Disponível em: https://www.researchgate.net/publication/373543929_Text_Detection_and_Recognition_from_the_Scene_Images_Using_RCNN_and_EasyOCR. Acesso em: 04 out. 2025.

CHOI, Sugi; KIM, Sunghwan; JUNG, Haiyoung. Optimized Faster R-CNN with Swintransformer for Robust Multi-Class Wildfire Detection. **Fire**, [S.L.], v. 8, n. 5, p. 180, 30 abr. 2025. MDPI AG. DOI: 10.3390/fire8050180. Disponível em: https://www.mdpi.com/2571-6255/8/5/180. Acesso em: 29 set. 2025.

CSURKA, Gabriela *et al*. **Semantic Image Segmentation**: two decades of research. [S.L.], fev. 2023. ArXiv. DOI: 10.48550/ARXIV.2302.06378. Disponível em: https://arxiv.org/abs/2302.06378. Acesso em: 30 set. 2025.

DALAL, N.; TRIGGS, B.. Histograms of Oriented Gradients for Human Detection. **2005 Ieee Computer Society Conference On Computer Vision And Pattern Recognition (Cvpr'05)**, [S.L.], v. 1, p. 886-893, jul. 2005. IEEE. DOI:10.1109/cvpr.2005.177. Disponível em: https://ieeexplore.ieee.org/document/1467360. Acesso em: 27 abr. 2025.

FAOUZI, Johann. Classic machine learning methods. In: COLLIOT, Olivier. **Machine Learning for Brain Disorders**. Paris: Cnrs, 2023. Cap. 2. Disponível em: https://link.springer.com/book/10.1007/978-1-0716-3195-9. Acesso em: 26 set. 2025.

GAIKWAD, Vrushab. OpenCV: a comprehensive review and applications in computer vision. **International Journal Of Advanced Research In Science, Communication And Technology (Ijarsct).** [S. L], p. 291-297. jul. 2023. Disponível em: https://ijarsct.co.in/Paper12140.pdf. Acesso em: 01 out. 2025.

GHODRATI, Amir; DIBA, Ali; PEDERSOLI, Marco; TUYTELAARS, Tinne; VAN GOOL, Luc. DeepProposals: hunting objects and actions by cascading deep convolutional layers. **International Journal Of Computer Vision**, [S.L.], v. 124, n. 2, p. 115-131, 15 mar. 2017. Springer Science and Business Media LLC. DOI:10.1007/s11263-017-1006-x. Disponível em: https://link.springer.com/article/10.1007/s11263-017-1006-x. Acesso em: 27 set. 2025.

HAFIZ, Abdul Mueed; BHAT, Ghulam Mohiuddin. **A Survey on Instance Segmentation**: state of the art., [S.L.], 2020. ArXiv. DOI: 10.48550/ARXIV.2007.00047. Disponível em: https://arxiv.org/abs/2007.00047. Acesso em: 30 set. 2025.

JIANG, Yuchen *et al*. Quo vadis artificial intelligence? **Discover Artificial Intelligence**, [S.L.], v. 2, n. 1, 7 mar. 2022. Springer Science and Business Media LLC. DOI:10.1007/s44163-022-00022-8. Disponível em: https://link.springer.com/article/10.1007/s44163-022-00022-8. Acesso em: 22 mar. 2025.

JOSHI, Prateek Joshiprateek *et al*. **OpenCV By Example**: enhance your understanding of computer vision and image processing by developing real-world projects. Birmingham: Packt Publishing, 2016.

KAGGLE. *Datasets*. Disponível em: https://www.kaggle.com/datasets. Acesso em: 28 ago. 2025.

KANG, Shizhao *et al*. **Object Detection YOLO Algorithms and Their Industrial Applications**: Overview and Comparative Analysis. 2025, Basel. [S.L.]: Mdpi, 2025. DOI: https://doi.org/10.3390/electronics14061104 Disponível em: https://www.mdpi.com/2079-9292/14/6/1104. Acesso em: 26 set. 2025.

KIRILLOV, Alexander *et al*. **Panoptic Segmentation**. [S.L.], jan. 2018. DOI: 10.48550/ARXIV.1801.00868. Disponível em: https://arxiv.org/abs/1801.00868. Acesso em: 29 set. 2025.

KUZNETSOVA, Alina et al. The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale. **International Journal of Computer Vision**, v. 128, n. 7, p. 1956-1981, 13 mar. 2020. DOI: 10.1007/s11263-020-01316-z. Disponível em: https://link.springer.com/article/10.1007/s11263-020-01316-z. Acesso em: 29 ago. 2025.

LAROCA, Rayson *et al*. A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector. **2018 International Joint Conference On Neural Networks (Ijcnn)**, [S.L.], p. 1-10, jul. 2018. Disponível em: https://web.inf.ufpr.br/vri/databases/ufpr-alpr/. Acesso em: 16 maio 2025.

LI, Shuangmei; HUANG, Chengning. Using convolutional neural networks for image semantic segmentation and object detection. **Systems And Soft Computing**, [S.L.], v. 6, n. 3520, p. 200172-1519519, dez. 2024. Elsevier BV. DOI: 10.1016/j.sasc.2024.200172. Disponível em: https://www.sciencedirect.com/science/article/pii/S2772941924001017. Acesso em: 27 set. 2025.

LIN, Tsung-Yi *et al*. **Microsoft COCO**: common objects in context. [S.L.], p. 740-755, fev. 2014. ArXiv. DOI: 10.48550/ARXIV.1405.0312. Disponível em: https://arxiv.org/abs/1405.0312. Acesso em: 27 set. 2025.

LIU, Huanyu *et al*. **An End-to-End Network for Panoptic Segmentation**. , [S.L.],, mar. 2019. DOI:10.48550/arXiv.1903.05027. Disponível em: https://arxiv.org/abs/1903.05027. Acesso em: 01 out. 2025.

LIU, Yajing *et al*. **Unbiased Faster R-CNN for Single-source Domain Generalized Object Detection**, [S.L.], v. 211, n. 200, p. 327-345, maio 2024. ArXiv. DOI:

10.48550/ARXIV.2405.15225. Disponível em: https://arxiv.org/pdf/2405.15225. Acesso em: 29 set. 2025.

LOWE, David G.. Distinctive Image Features from Scale-Invariant Keypoints. **International Journal Of Computer Vision**, [S.L.], v. 60, n. 2, p. 91-110, nov. 2004. Springer Science and Business Media LLC. http://dx.doi.org/10.1023/b:visi.0000029664.99615.94. Disponível em: https://link.springer.com/article/10.1023/B:VISI.0000029664.99615.94#article-inf. Acesso em: 27 abr. 2025.

MAGDY, Andrew *et al*. Lightweight faster R-CNN for object detection in optical remote sensing images. **Scientific Reports**, [S.L.], v. 15, n. 1, p. 210-255, 9 maio 2025. Springer Science and Business Media LLC. DOI: 10.1038/s41598-025-99242-y. Disponível em: https://www.nature.com/articles/s41598-025-99242-y. Acesso em: 30 set. 2025.

MALINSKI, Kamil; OKARMA, Krzysztof. Analysis of Image Preprocessing and Binarization Methods for OCR-Based Detection and Classification of Electronic Integrated Circuit Labeling. **Electronics**, [S.L.], v. 12, n. 11, p. 2449, 29 maio 2023. MDPI AG. DOI: 10.3390/electronics12112449. Disponível em: https://www.mdpi.com/2079-9292/12/11/2449. Acesso em: 02 out. 2025.

MATHWORKS. **MATLAB**: *Version 9.10.0 (R2021a)*. Natick, MA: The MathWorks Inc., 2023. Disponível em: https://www.mathworks.com. Acesso em: 25 out. 2023.

MUKHOPADHYAY, Sayan; SAMANTA, Pratip. Unsupervised Learning: clustering. **Advanced Data Analytics Using Python**, [S.L.], p. 81-113, 26 nov. 2022. DOI: 10.1007/978-1-4842-8005-8_4. Disponível em: https://link.springer.com/chapter/10.1007/978-1-4842-8005-8_4. Acesso em: 27 set. 2025.

MÜLLER, Andreas C.; GUIDO, Sarah. **Introduction to Machine Learning with Python**: A guide for Data Scientists.Sebastopol: O'reilly Media, 2017. 392 p.

NEUDECKER, Clemens *et al*. A survey of OCR evaluation tools and metrics. **The 6Th International Workshop On Historical Document Imaging And Processing**, [S.L.], p. 13-18, 5 set. 2021. DOI: 10.1145/3476887.3476888. Disponível em: https://dl.acm.org/doi/10.1145/3476887.3476888. Acesso em: 22 out. 2025.

NIXON, Mark; AGUADO, Alberto. **Feature Extraction and Image Processing for Computer Vision**. 4. ed. [S. L.]: Academic Press, 2019. 650 p.

OPENCV. **Open Source Computer Vision Library Docmentation**. 2025. Disponível em:https://opencv.org/. Acesso em: 26 abr. 2025.

Prefeitura de São Paulo. **Programa Smart Sampa**: o maior programa de câmeras de segurança da cidade de São Paulo. O maior programa de câmeras de segurança da cidade de São Paulo. 2025. Disponível em: https://capital.sp.gov.br/web/seguranca_urbana/w/smart-sampa-2. Acesso em: 17 abr. 2025

Prefeitura de São Paulo. ***Smart Sampa**: tecnologia e inteligência artificial reforçam segurança em São Paulo*. Estúdio Folha, São Paulo, 19 mar. 2025. Disponível em: https://estudio.folha.uol.com.br/prefeitura-de-saopaulo/2025/03/smart-sampa-tecnologia-e-inteligencia-artificial-reforcam-seguranca-em-sao-paulo.shtml. Acesso em: 15 abr. 2025.

PYTORCH. PyTorch: An Open Source Machine Learning Framework. 2025. Disponível em: https://pytorch.org/. Acesso em: 26 abr. 2025.

RAJ, Ravi; KOS, Andrzej. An Extensive Study of Convolutional Neural Networks: applications in computer vision for improved robotics perceptions. **Sensors**, [S.L.], v. 25, n. 4, p. 1033, 9 fev. 2025. MDPI AG. DOI: http://dx.doi.org/10.3390/s25041033. Disponível em: https://www.mdpi.com/1424-8220/25/4/1033. Acesso em: 27 set. 2025.

RAO, Tabish *et al*. Comprehensive study on automatic number plate recognition with python using OpenCV and EasyOCR. **Challenges In Information, Communication And Computing Technology**, [S.L.], p. 501-505, 19 nov. 2024. CRC Press. DOI: 10.1201/9781003559092-86. Disponível em: https://www.researchgate.net/publication/385959609_Comprehensive_study_on_automatic_number_plate_recognition_with_python_using_OpenCV_and_EasyOCR. Acesso em: 05 out. 2025.

REDMON, Joseph *et al*. You Only Look Once: unified, real-time object detection. **2016 IEEE Conference On Computer Vision And Pattern Recognition (CVPR)**, [S.L.], p. 779-788, jun. 2016. IEEE. DOI:10.1109/cvpr.2016.91. Disponível em: https://ieeexplore.ieee.org/document/7780460. Acesso em: 17 mar. 2025.

REN, Shaoqing; HE, Kaiming; GIRSHICK, Ross; SUN, Jian. Faster R-CNN: towards real-time object detection with region proposal networks.*In*: CORTES, C. et al. (Ed.) **Advances in Neural Information Processing Systems**, Curran Associates, Inc., 2015. v. 28.DOI: 10.48550/arXiv.1506.01497. Disponível em: https://arxiv.org/pdf/1506.01497. Acesso em: 17 mar. 2025.

RIABKO, Artem; AVERYANOVA, Yuliya. **Comparative analysis of SIFT and SURF methods for local feature detection in satellite imagery**. In: CMSE'24: INTERNATIONAL WORKSHOP ON COMPUTATIONAL METHODS IN SYSTEMS ENGINEERING, 2., 2024, Kyiv. Proceedings [...] **.** Kyiv: Ceur, 2024. Disponível em: https://ceur-ws.org/Vol-3732/paper02.pdf. Acesso em: 30 set. 2025.

SCIKIT-IMAGE. Image processing in Python — scikit-image. 2025. Disponível em: https://scikit-image.org/. Acesso em: 26 abr. 2025.

SHORTEN, Connor; KHOSHGOFTAAR, Taghi M.. A survey on Image Data Augmentation for Deep Learning. **Journal Of Big Data**, [S.L.], v. 6, n. 1, p. 1-48, 6 jul. 2019. Springer Science and Business Media LLC. DOI:10.1186/s40537-019-0197-0. Disponível em: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0. Acesso em: 27 abr. 2025.

SINGH, Shweta; YADAV, Aayan; JAIN, Jitesh; SHI, Humphrey; JOHNSON, Justin; DESAI, Karan. Benchmarking Object Detectors with COCO: a new path forward., [S.L.], 2024. ArXiv. DOI: 10.48550/ARXIV.2403.18819. Disponível em: https://arxiv.org/abs/2403.18819. Acesso em: 27 set. 2025.

SZELISKI, Richard. **Computer Vision**: algorithms and applications. 2. ed. [S. L.]: Springer, 2022. 1232 p.

TENSORFLOW. TensorFlow: An end-to-end open source machine learning platform. 2025. Disponível em: https://tensorflow.org/. Acesso em: 26 abr. 2025.

THIRUNAVUKKARASU, Krishnika; SHANKAR, Haritha. A Computer Vision Powered OCR Framework for Extracting Tabular Data from Scanned PDFs. **International Journal For Research Trends And Innovation(Ijrti)**, Chennai, v. 10, p. 126-130, abr. 2025. Disponível em: https://www.ijrti.org/papers/IJRTI2504315.pdf. Acesso em: 04 out. 2025.

XIE, Yuxiang; YAN, Jie; KANG, Lai; GUO, Yanming; ZHANG, Jiahui; LUAN, Xidao. FCT: fusing cnn and transformer for scene classification. **International Journal Of Multimedia Information Retrieval**, [S.L.], v. 11, n. 4, p. 611-618, 15 set. 2022. DOI: 10.1007/s13735-022-00252-7. Disponível em:

https://www.researchgate.net/publication/363584255_FCT_fusing_CNN_and_transformer_for_scene_classification. Acesso em: 13 out. 2025.

YANG, Zhengxian *et al*. A Review of Document Binarization: main techniques, new challenges, and trends. **Electronics**, [S.L.], v. 13, n. 7, p. 1394, 7 abr. 2024. MDPI AG. DOI: 10.3390/electronics13071394. Disponível em: https://www.mdpi.com/2079-9292/13/7/1394. Acesso em: 02 out. 2025.

ZHANG, Linjiang; WANG, Peng; LI, Hui; LI, Zhen; SHEN, Chunhua; ZHANG, Yanning. **A Robust Attentional Framework for License Plate Recognition in the Wild**. [S.L.], 2020. DOI:10.48550/ARXIV.2006.03919. Disponível em: https://arxiv.org/abs/2006.03919. Acesso em: 13 out. 2025.

ZHAO, Zhong-Qiu *et al*. Object Detection With Deep Learning: a review. **IEEE Transactions On Neural Networks And Learning Systems**, [S.L.], v. 30, n. 11, p. 3212-3232, nov. 2019. Institute of Electrical and Electronics Engineers (IEEE). http://dx.doi.org/10.1109/tnnls.2018.2876865.

ZHOU, Tianfei *et al*. **Image Segmentation in Foundation Model Era**: a survey. [S. L.], 23 ago. 2024. DOI: 10.48550/arXiv.2408.12957. Disponível em: http://arxiv.org/abs/2408.12957. Acesso em: 27 set. 2025.